



Review

Comprehensive review of smart contract and DeFi security: Attack, vulnerability detection, and automated repair

Peng Qian^{a,b}, Rui Cao^c, Zhenguang Liu^a, Wenqing Li^a, Ming Li^b, Lun Zhang^b,
Yufeng Xu^b, Jianhai Chen^{a,*}, Qinming He^a

^a College of Computer Science and Technology, Zhejiang University, Hangzhou, China

^b Goplus Security, Hangzhou, China

^c College of Computer Science and Technology, Zhejiang Gongshang University, Hangzhou, China

ARTICLE INFO

Keywords:

Decentralized finance (DeFi)

Smart contract

Blockchain

Vulnerability detection

Automated repair

ABSTRACT

Decentralized Finance (DeFi) is emerging as a peer-to-peer financial ecosystem. Currently, the substantial value locked in DeFi protocols makes them attractive targets for malicious attacks, which have resulted in huge losses and pose a severe threat to the overall security of the DeFi ecosystem. While researchers have proposed a variety of strategies to safeguard DeFi and smart contracts, a comprehensive study of these efforts remains elusive, leaving a critical gap in understanding how to improve the security posture of DeFi and smart contracts. To fill this gap, this paper provides a comprehensive review of advances in the field of DeFi security, with a focus on bug detection, attack hunting, risk assessment, and automated repair. First, we dive into the complexity of DeFi protocol security by analyzing prevalent issues and challenges. Specifically, we summarize the security threats in DeFi, and then scrutinize a spectrum of DeFi attack incidents, categorizing them into six types. We then present an empirical review of 9 existing DeFi security tools that can either identify vulnerabilities and attacks against DeFi protocols or assess their security risks. In addition, 38 traditional smart contract bug detection tools are also studied in our work. Experiments have been conducted to evaluate the effectiveness of these tools. Furthermore, we conduct a preliminary investigation of 8 existing automated repair tools and briefly discuss their suitability for DeFi smart contract patching. To ensure the utmost utility of this research across a wide audience, we highlight the outstanding issues and challenges of the DeFi ecosystem. As a side contribution, we make available a dataset containing 99 DeFi protocols (including 7340 DeFi smart contracts) that address six types of DeFi attacks, hoping to facilitate the DeFi community.

1. Introduction

Blockchain and its groundbreaking applications, such as Bitcoin (Nakamoto, 2008) and smart contracts (Zou et al., 2019), are revolutionizing various industries and ushering in a new era of decentralized solutions. In essence, blockchain is a replicated and distributed ledger shared by all participating nodes within a peer-to-peer network, all operating under a consensus protocol (Liu et al., 2021). Consensus mechanisms constitute the backbone of blockchain technology, attracting extensive scrutiny from academia and industry. The consensus mechanism continues to expand with the evolution of blockchain technology, encompassing prominent protocols, e.g., PoW (Gervais et al., 2016), PoS (King & Nadal, 2012), and DPoS (DPO, 2014), paving the way for

the emergence of numerous innovations of consensus mechanisms (Liu, 2021).

Smart contracts are computer programs that run on a blockchain ecosystem. They encode pre-designed terms and conditions into executable code, ensuring that these rules are rigorously adhered to during execution. Smart contracts play a key role in enabling the automatic execution of these terms, boosting the development of sophisticated decentralized applications (Chen, Xu, Shi, Zhao, & Zhao, 2018). Ethereum, one of the leading blockchain platforms, has witnessed the deployment of tens of millions of smart contracts, which enable a wide range of applications spanning various industries, including but not limited to wallets (Chen & Ko, 2019), gambling (Hu et al., 2021), healthcare (Agbo, Mahmoud, & Eklund, 2019), supply

* Corresponding author.

E-mail addresses: pqian@zju.edu.cn (P. Qian), primercui@gmail.com (R. Cao), liuzhenguang2008@gmail.com (Z. Liu), lwqfighting@zju.edu.cn (W. Li), Mike@gopluslabs.io (M. Li), Allen@gopluslabs.io (L. Zhang), eskil@gopluslabs.io (Y. Xu), chenjh919@zju.edu.cn (J. Chen), hqm@zju.edu.cn (Q. He).

<https://doi.org/10.1016/j.eswa.2025.128431>

Received 24 September 2023; Received in revised form 13 May 2025; Accepted 30 May 2025

Available online 5 June 2025

0957-4174/© 2025 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

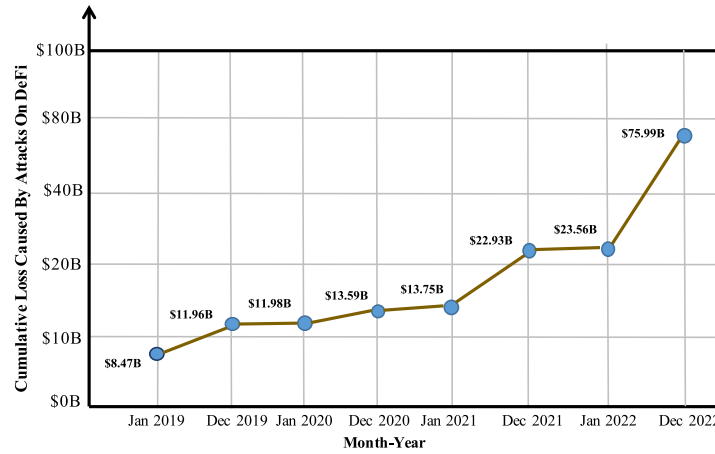


Fig. 1. The cumulative losses caused by attacks targeting DeFi protocols. The data comes from the REKT database (REKT, 2023).

chain management (Moosavi, Naeni, Fathollahi-Fard, & Fiore, 2021), and cross-industry finance (Williams, 2020). The proliferation of smart contracts underscores their transformative potential to revolutionize traditional business processes and interactions.

Recently, there has been a remarkable surge in the adoption and prominence of decentralized finance (DeFi) applications (Jensen, von Wachter, & Ross, 2021). This surge has led to an astounding increase in the total value locked (TVL) within the DeFi ecosystem, reaching heights as staggering as \$248.84 billion. This can be attributed to the rapid expansion of DeFi applications, which have demonstrated their potential to extend the utility of blockchain technology beyond simple value transfers to encompass a variety of complex financial services (Amler et al., 2021). Prominent DeFi protocols have emerged that offer an impressive array of decentralized financial services, including lending and borrowing, portfolio management, decentralized exchanges, and derivatives markets. What sets these DeFi applications apart is their ability to deliver services without relying on trusted intermediaries. This decentralized nature empowers users to engage in financial activities with increased transparency, security, and autonomy, ultimately reshaping the landscape of the financial industry.

1.1. Motivation I: Understanding DeFi vulnerabilities and attacks

While DeFi has shown significant progress, it remains in a relatively nascent stage, and the emerging DeFi ecosystem still exhibits some rough edges, leaving ample room for security vulnerabilities and attacks (Qin, Zhou, Livshits, & Gervais, 2021c). These security concerns have been highlighted by a number of high-profile incidents. For example, in February 2020, the well-known DeFi protocol bZx fell victim to two consecutive attacks (Oosthoek, 2021). By exploiting the inherent logic flaw within bZx, attackers are able to perform arbitrage maneuvers that result in the theft of over \$8 million worth of ETH. In April 2020, cybercriminals hacked into the DeFi protocol UniswapV1, exploiting a reentrancy vulnerability to steal 1278 ETH (Group, 2023). A more recent instance occurred in October 2021 when the DeFi protocol Cream.Finance fell prey to an external attack, resulting in losses exceeding \$130 million (InspeX, 2023).

Most notably, the DeFi cross-chain project Poly Network (Wikipedia, 2023) and the NFT game Ronin (Hayward, 2023) experienced devastating hacker attacks, resulting in losses of \$611 million and \$622 million, respectively. These attacks represent the most severe breaches in the DeFi area at the time of writing. Alarming statistics from the REKT Database (REKT, 2023) reveal that DeFi has suffered total losses of \$77.1 billion due to scams, hacks, and exploits, with only \$6.5 billion recovered. Fig. 1 illustrates the cumulative financial losses caused by external attacks on DeFi protocols

over the past three years, showing a significant impact of these frequent attacks on the DeFi ecosystem. Not only have these incidents caused huge financial loss, but also they have posed a considerable hindrance to its development and broader adoption.

Notwithstanding recent efforts that have been directed towards investigating DeFi security issues (Amler et al., 2021; Wang et al., 2021b), a comprehensive and well-structured overview of this critical topic is still conspicuously absent. Currently, most advances in DeFi (Chohan, 2021; Qin, Zhou, Afonin, Lazzaretti, & Gervais, 2021a) tend to focus on financial applications and services, often overlooking an in-depth exploration of the various vulnerabilities and attacks that target the DeFi protocols. *Motivated by this observation, we aim to bridge this gap and provide a holistic understanding of the spectrum of DeFi vulnerabilities and attacks. By doing so, we hope to provide valuable insights that can help strengthen the resilience of the DeFi community against these threats.*

1.2. Motivation II: Investigating security tools for DeFi protocols

A number of efforts (Perez & Livshits, 2021; Xue et al., 2020) have been made to automatically detect defects in smart contracts, primarily focusing on static and dynamic analysis techniques. For example, Slither (Feist, Grieco, & Groce, 2019) transforms smart contracts into an intermediate representation and performs static taint analysis to identify bugs. ContractFuzzer (Jiang, Liu, & Chan, 2018) applies fuzzing techniques to smart contracts and exposes vulnerabilities by monitoring their runtime behavior during the fuzzing process. *It should be noted, however, that most of these approaches are tailored for traditional smart contracts, which typically comprise a single contract. Their effectiveness in performing vulnerability analysis on complex DeFi protocols, which consist of multiple interconnected smart contracts, has yet to be thoroughly investigated.*

Recent advances (Cao, Zou, & Cheng, 2021; Wang et al., 2021a) have been dedicated to detecting vulnerabilities and attacks through the analysis of transactions within DeFi protocols. These efforts involve extracting transaction patterns to uncover potential flaws and attacks. For example, ProMutator (Wang, Wu, Liang, Hsieh, & Hsiao, 2021c) focuses on detecting price manipulation attacks by reconstructing DeFi usage patterns based on historical transactions. DeFiRanger (Wu et al., 2021), on the other side, identifies price oracle bugs by analyzing high-level DeFi semantics in transaction data. *Despite the growing interest in DeFi security, there is currently a lack of systematic overview to assess the existing DeFi security tools and evaluate their effectiveness. This research gap hinders the ability to fully understand and address the security challenges facing the DeFi ecosystem.*

Another important line of research focuses on fixing bugs in DeFi protocols and smart contracts. For instance, Tolmach, Li, and Lin (2023)

have pioneered an automated repair methodology specifically tailored for DeFi protocols, which is able to rectify violations of functional specifications expressed as properties while ensuring robust correctness guarantees. SGuard (Nguyen, Pham, & Sun, 2021a) designs specific fixing patterns for different vulnerabilities and patches smart contracts to be free of those bugs. *It is worth noting that despite the growing significance of these automated repair tools for DeFi smart contracts, there is currently a dearth of comprehensive reviews evaluating the existing tools in this domain. Such a review would serve as a valuable resource for the research community and practitioners alike, in the ongoing quest to enhance the security and reliability of blockchain-based DeFi protocols.*

1.3. Our work and contribution

In this paper, our goal is to provide a comprehensive understanding of various DeFi vulnerabilities and attacks while also conducting an empirical review of state-of-the-art security tools for DeFi protocols and smart contracts. *On one hand*, we identify common DeFi vulnerabilities and organize them into a taxonomy. Furthermore, we study 57 documented DeFi attack incidents. From these cases, we distill and categorize six common types of DeFi attacks. To enhance understanding, we provide concrete examples that illustrate the attack process for each category of DeFi attack. *On the other hand*, we explore the principles of existing techniques that can either detect DeFi vulnerabilities and attacks, or assess risk, or automate the repair of DeFi protocols, with the analysis of their strengths, weaknesses, and workflows. In particular, we carry out experiments to evaluate their efficacy in identifying vulnerabilities and attacks within DeFi protocols. Additionally, we investigate 38 tools designed to detect vulnerabilities in smart contracts, and perform experiments to assess their effectiveness when applied to complex DeFi protocols. Finally, we highlight the limitations of existing security tools in addressing the complexity of DeFi protocols and provide novel insights. Our study culminates in the identification of several open issues and challenges within the DeFi ecosystem that beckon future research endeavors for resolution and advancement.

Contributions. The key contributions of this paper can be summarized as:

- This work, to our knowledge, is the first to present a comprehensive review of various DeFi vulnerabilities and attacks and a systematic evaluation of the state-of-the-art security tools for DeFi protocols and smart contracts, empirically revealing both capabilities and limitations of existing methods.
- We review common DeFi vulnerabilities and organize them into a taxonomy. In light of 57 reported DeFi attack incidents, we summarize six categories of DeFi vulnerabilities or attacks. Further, we also construct a benchmark dataset, which contains a total of 7340 DeFi smart contracts. To facilitate the community, we have released this dataset for public use at <https://github.com/Messi-Q/DeFi-Protocol>.
- We empirically study 55 security tools for DeFi protocols, involving vulnerability detection, attack hunting, risk assessment, and automated repair. We also discuss several open issues and challenges in the DeFi ecosystem that need to be tackled by future research.

2. Background

Before diving into the key components, please allow us to introduce the required background on blockchain, smart contract, and decentralized finance.

2.1. Blockchain in a Nutshell

Blockchain, originally popularized as the underlying technology for Bitcoin, is a distributed ledger maintained in a decentralized fashion. In the context of a public blockchain network, it provides an open and transparent infrastructure that is not owned or controlled by a central

authority. To elaborate further, the distributed ledger comprises a series of blocks, each of which contains a set of recorded transactions. These blocks are linked sequentially, forming a chain-like structure. Once a new block is validated by miners adhering to a consensus protocol, it is appended to the chain (Bamakan, Motavali, & Bondarti, 2020). Each miner maintains a duplicate transaction ledger, thereby granting the blockchain its decentralized and immutable attributes. This means that no entity can manipulate or erase data once it is recorded in the ledger.

2.2. Ethereum and smart contract

Ethereum stands as one of the most prominent blockchain platforms, distinguishing itself as the pioneer in introducing smart contract functionality (Buterin et al., 2019). Essentially, smart contracts represent digital agreements with coded rules that can be enforced autonomously without the need for a trusted intermediary. Once deployed, smart contracts are uploaded to the blockchain and distributed to all participating nodes. Smart contracts have the ability to enforce a wide range of rules for manipulating of digital assets. These rules are strictly followed during execution, adhering to the logic of ‘code is law’ (Liu et al., 2023b). Despite Ethereum’s rise as one of the most influential blockchains, it has faced a significant number of security vulnerabilities within its smart contracts (Perez & Livshits, 2019). Given that a DeFi protocol usually consists of multiple interconnected smart contracts, it has inevitably become attractive targets for potential attackers. As a result, these vulnerabilities have led to significant financial losses within the DeFi ecosystem.

2.3. DeFi: Decentralized finance

Decentralized Finance (DeFi) is a thriving peer-to-peer financial system that harnesses the power of blockchain technology (Schär, 2021). With the widespread adoption of smart contracts, the concept of decentralized finance has truly come into its own, with the total value locked in DeFi exceeding \$200 billion (Qin et al., 2021a). DeFi leverages the permissionless nature of blockchain networks, allowing anyone to propose innovative financial contracts and enabling anyone to interact with, transfer assets to, and withdraw assets from those contracts, provided they adhere to the immutable rules encoded in them. Currently, a diverse array of financial services is making the transition to the DeFi ecosystem. To provide a clear understanding of what DeFi encompasses and what it can achieve, we provide a high-level overview of DeFi in Fig. 2. These services encompass lending and borrowing (Bartoletti, Chiang, & Lafuente, 2021b), decentralized exchanges (Bartoletti, Chiang, & Lluch-Lafuente, 2021a), portfolio management (Bartoletti, Chiang, & Lluch-Lafuente, 2021d), derivatives markets (Bartoletti, Chiang, & Lafuente, 2021c), stablecoin, aggregator applications, and many others (Abdulhakeem, Hu et al., 2021; Amler et al., 2021).

3. Overview of DeFi vulnerability and attack

DeFi has undeniably experienced an incredible surge in popularity, penetrating various industries. However, the increasing complexity of DeFi protocols has led to a continuous wave of security vulnerabilities. Meanwhile, the rapid growth and the considerable wealth locked in the DeFi protocols have attracted a slew of external attacks, posing significant security threats to the entire DeFi ecosystem (Chaliasos et al., 2024; Wang, Qin, Minh, & Gervais, 2022c). While many DeFi attack events have received attention, there remains a lack of a well-structured review that can provide an in-depth analysis of these vulnerabilities and attacks.

To this end, we strive to fill this gap by first identifying common vulnerabilities in DeFi protocols and organizing them into a taxonomy. Furthermore, we provide a comprehensive review of various DeFi attacks, with the goal of inspiring and enlightening communities. It is critical to clarify that while numerous studies have explored various flaws within traditional smart contracts, e.g., *timestamp dependency* and *unhandled*

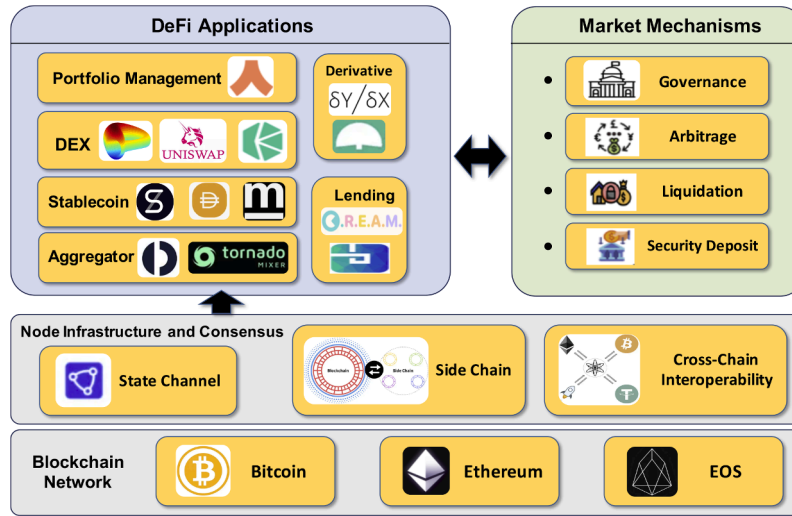


Fig. 2. DeFi builds upon a distributed blockchain network, enabling various DeFi applications, such as portfolio management, decentralized exchanges, derivatives, lending and borrowing, stablecoin, and aggregator.

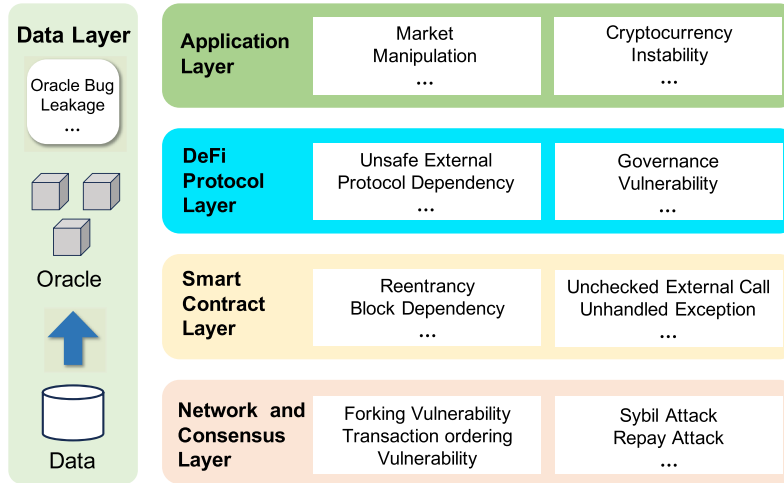


Fig. 3. An overview of DeFi vulnerabilities spanning different levels of the DeFi ecosystem.

exception (Chu et al., 2023; Qian et al., 2022), our focus in this work is primarily on vulnerabilities and attacks that specifically target DeFi protocols. In what follows, we provide detailed insights into the classification of DeFi vulnerabilities and each type of DeFi attack, offering a rigorous examination of their intricacies and implications.

3.1. Analysis of DeFi vulnerabilities

To provide a comprehensive perspective on security threats in DeFi, we classify vulnerabilities across five layers of the DeFi ecosystem, including the data, consensus, smart contract, DeFi protocol, and application layers (Li, Bu, Li, & Chen, 2022a; Zhou et al., 2023). Fig. 3 provides a high-level overview of this multi-layered taxonomy.

3.1.1. Data layer

The data layer governs the storage and accessibility of key information within DeFi systems, e.g., account balances, interest rates, and price feeds. Vulnerabilities at this level could compromise the integrity, confidentiality, and availability of data, ultimately threatening the correctness of higher-layer protocol behavior.

Oracle Vulnerability. Oracles act as bridges between blockchains and external systems, supplying off-chain data to smart contracts. If an or-

acle is compromised or the data it fetches is manipulated, protocols relying on that data may execute invalid logic. Notable attacks include the 2020 bZx oracle manipulation (Kistner, 2023b), where attackers exploited low-liquidity assets on decentralized exchanges (DEXs) to feed incorrect prices to the oracle, resulting in over \$1 million in losses.

Data Leakage Vulnerability. Sensitive data leakage - such as private keys, off-chain credentials, or unintended metadata - can result in unauthorized access or misuse. For example, misconfigured IPFS gateways or unencrypted off-chain storage could expose user identities or internal protocol logic, posing risks to both security and privacy.

3.1.2. Consensus layer

The consensus layer ensures transaction ordering and state finality in the blockchain. DeFi protocols are exposed to risks when attackers exploit consensus properties to gain unfair advantages or cause system inconsistencies.

Forking Vulnerability. Forks occur when a blockchain splits into two or more separate chains due to changes in consensus rules or disagreements within the network (Daian et al., 2020). In DeFi, forks can create inconsistencies in contract state or execution history, leading to replay attacks or asset duplication. For instance, the Ethereum-Ethereum Classic split

created duplicated token balances on both chains, which some attackers exploited for double spending.

Transaction Ordering Vulnerability. Attackers can manipulate the order of transactions via frontrunning, backrunning, or sandwich attacks (Wang, Partick, Yao, Lu, & Wattenhofer, 2022b). For example, a frontrunner may detect a large DEX trade and pay a higher gas fee to execute their transaction beforehand, profiting from the victim's impact on the market price. Tools such as Flashbots have revealed that miner extractable value exploitation has become widespread (Zhou, Qin, Torres, Le, & Gervais, 2021c).

Sybil Attack. In voting-based or identity-sensitive DeFi systems, attackers may create multiple pseudonymous accounts to gain disproportionate influence. This undermines consensus integrity and can lead to malicious governance decisions, especially in protocols that rely on quadratic voting or airdrop mechanisms (Li et al., 2022b).

Replay Attack. If chains fail to implement replay protection, a transaction valid on one forked chain can be reused on another, leading to unintended asset transfer or loss. This issue is relevant in systems that undergo hard forks or operate across Layer-2 solutions without domain separation (Cao et al., 2021).

3.1.3. Smart contract layer

Previous studies (He et al., 2020; Qian et al., 2022) have identified a number of smart contract vulnerabilities, and we have compiled typical ones that attackers might exploit to gain a profit in Table 4. Below, we highlight three vulnerabilities that commonly occur in DeFi protocols.

Reentrancy. A reentrancy vulnerability occurs when a smart contract transfers funds to an external contract, and the external contract is able to call back into the original function, or another function of the sender contract, before the initial execution is complete (Ye, Ma, Lin, Sui, & Xue, 2020). This recursive calling can potentially result in logic errors, state inconsistencies, or unintended withdrawals of funds by the external contract.

Block Information Dependency. Block information refers to the properties of blocks in the blockchain, such as block hash, block timestamp. They are commonly used as random seeds due to the inherent randomness (Qian et al., 2023a). However, malicious attackers may exploit the block information to manipulate or predict pseudo-random number generation, leading to illicit profits.

Unchecked External Call. A contract that calls an external contract or a set of external contracts forms an external contract call chain. If the return value of a call in the call chain is unchecked, indicating that an exception was thrown, the first external call in the call chain will not find and handle that exception.

3.1.4. DeFi protocol layer

Notably, vulnerabilities in the DeFi protocol layer may resemble financial market manipulation rather than traditional system vulnerabilities (Zhou et al., 2023), often stemming from protocol design flaws.

Unsafe External Protocol Dependency. The protocol layer comprises a set of DeFi protocols that are deployed and built on top of the smart contract layer. The vulnerability of unsafe external protocol dependency in DeFi refers to the risks associated with relying on external protocols or systems that may lack adequate security or trustworthiness. This vulnerability arises when DeFi projects integrate with external protocols, oracles, or data sources without performing validation or assurance of their security and reliability. If a DeFi protocol depends on an external protocol for certain functionalities, any vulnerabilities or weaknesses in these external dependencies can be exploited to compromise the security and integrity of the DeFi protocol.

Governance Vulnerability. Governance vulnerabilities in DeFi protocols refer to flaws in decentralized decision-making mechanisms that can result in centralization, mismanagement, or exploitation (Liu, Lu, Zhu, Paik, & Staples, 2023a). These issues often stem from token-weighted voting systems or automated proposal executions lacking adequate safeguards. When a small group of participants controls a majority of governance tokens, they can unilaterally influence protocol decisions, undermining decentralization and user trust.

Real-world incidents highlight the severity of such risks. The 2016 DAO attack combined a reentrancy vulnerability with inadequate governance protections, enabling the attacker to recursively drain funds and ultimately causing the loss of over 3.6 million ETH. This incident prompted a contentious Ethereum hard fork to recover the stolen assets. More recently, Compound's Proposal 062 introduced a logic flaw that resulted in excessive COMP token distribution, attributed to insufficient auditing and the absence of emergency pause mechanisms. Such governance flaws could have been exploited further to introduce malicious code or redirect funds through improperly vetted proposals.

3.1.5. Application layer

It is worth noting that vulnerabilities in the DeFi application layer often stem from various market elements, including cryptocurrency prices, liquidity conditions, and more. Below, we introduce two typical instances.

Market Manipulation. Market manipulation in DeFi refers to the susceptibility of decentralized financial markets to being manipulated for illicit gain. It usually involves a deliberate attempt to interfere with the free and fair operation of the market, with the intent of profiting at the expense of other participants. For example, traders may exploit advanced knowledge of pending transactions to execute trades ahead of other market participants, profiting from the price movements that result from the anticipated trades.

Cryptocurrency Instability. The inherent volatility and instability of cryptocurrency prices expose potential flaws in DeFi. Volatility affects almost every aspect of DeFi - from lending platforms to automated market makers, yield farming, and more. The instability vulnerability in DeFi refers to the risk posed by the volatile nature of cryptocurrencies within DeFi systems. This vulnerability arises due to the inherent price fluctuations and uncertainty associated with cryptocurrencies, which can affect the stability and reliability of DeFi protocols. For example, DeFi protocols often rely on cryptocurrencies as collateral for loans or other financial activities. However, sudden declines in the value of these collateral assets can result in liquidations, margin calls, or defaults, leading to financial losses for users and instability within the DeFi ecosystem.

3.2. Analysis of real-World DeFi attack incidents

In this subsection, we further investigate real-world DeFi attack events that have occurred over the past three years. Specifically, we categorize these attacks into six major categories: *flash loan attack* (*pump and arbitrage*, *price manipulation*, *reentrancy*), *deflation token attack*, *sandwich attack*, and *rug pull attack*. It is worth noting that many DeFi attacks take advantage of the instability of the DeFi market to manipulate price fluctuations and steal profits.

3.2.1. Flash loan attack

As an emerging service in the DeFi ecosystem, flash loans offer users the convenience of obtaining non-collateralized loans. However, this convenience also opens the door for adversaries to launch malicious attacks using a significant amount of assets they do not own. Our examination of existing DeFi attacks reveals that approximately 50 % of DeFi attack incidents involve flash loans (Wang et al., 2021b), making it the most prevalent form of attack within the DeFi space. In particular, we categorize flash loan attacks into three main types: *pump and arbitrage*,

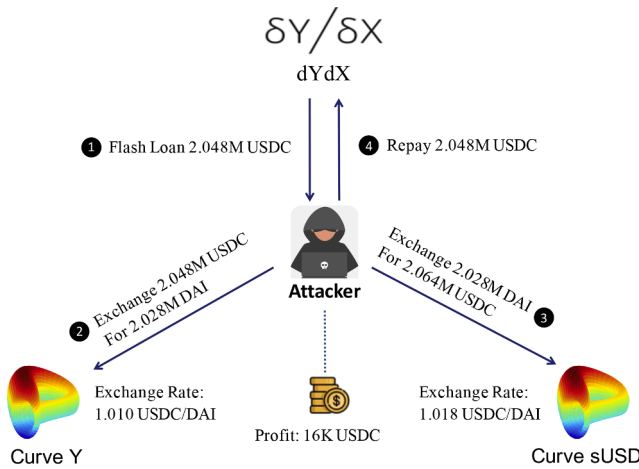


Fig. 4. A pump and arbitrage attack on the DeFi protocol – dYdX.

price manipulation, and *reentrancy*. For each type of flash loan attack, we aim to provide readers with an intuitive understanding of the characteristics of the attack. Subsequently, we deliver concrete examples to illustrate how these attacks unfold and operate.

Pump and Arbitrage. Arbitrage refers to the practice of making a profit by trading the same asset on different exchanges that offer different prices for that asset. In the context of DeFi, where market reactions to blockchain network events can be slower compared to the real-world market, attackers can exploit the inefficiencies to buy and sell an asset at different prices, thereby gaining financial advantages. Flash loans allow attackers to execute arbitrage strategies without the need for any pre-owned assets. When a price discrepancy is identified, attackers can immediately borrow a substantial amount of assets using a flash loan service and then achieve arbitrage. Note that the pump-and-arbitrage takes full advantage of the instability of cryptocurrency prices in DeFi markets.

As an illustrative example, we provide specific details of a pump and arbitrage attack targeting the DeFi protocol dYdX¹, as depicted in Fig. 4. Here is a step-by-step breakdown of the attack:

- The attacker initiates the attack by borrowing a flash loan of 2.048 million USDC from the dYdX exchange (Step 1).
- The attacker exchanges all borrowed USDC for 2.028 million DAI on another exchange, Curve Y, at an exchange rate of 1.010 USDC/DAI (Step 2).
- Next, the attacker exchanges all DAI for 2.064 million USDC on the third exchange, Curve sUSD, at an exchange rate of 1.018 USDC/DAI (Step 3).
- The attacker repays the initially borrowed 2.048 million USDC (Step 4).

With these exchanges, the attacker effectively generates an arbitrage profit of 16 thousand USDC (i.e., 16 thousand = 2.064 million - 2.048 million).

Price Manipulation. Price manipulation represents one of the most prevalent types of flash loan attacks. An attacker deliberately distorts reported prices and propagates this distortion using a vulnerable custom function, leading victims to receive inaccurate price data. The core of price manipulation attack is that the attacker lowers the exchange rate of a token by using a flash loan, and then benefits from the decreased exchange rate. This manipulation results in a lower exchange rate, allowing an attacker to profit from the price discrepancy.

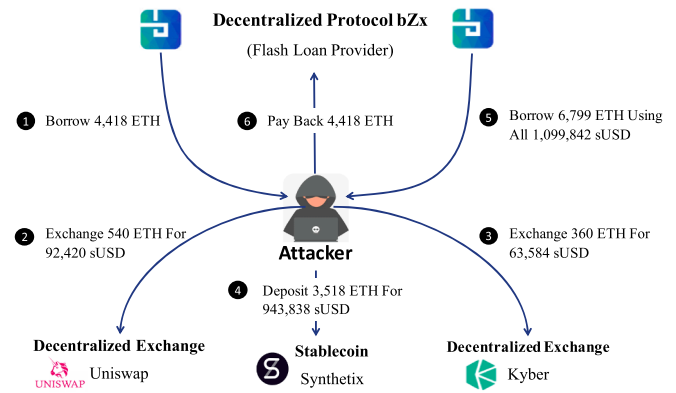


Fig. 5. A price manipulation attack on the DeFi protocol – bZx.

To provide a more detailed explanation of the price manipulation attack, we examine a real-world attack example targeting the DeFi protocol bZx², as shown in Fig. 5. In this case, the attacker exploits bZx's reliance on other DeFi protocols, including Uniswap and Kyber, to manipulate asset exchange rates and generate profits within a single transaction. In this attack, the attacker executes a sequence of six transactions, involving borrowing, exchanging, and repaying assets such as ETH and sUSD. Particularly, these transactions are bundled into a single transaction and executed in the following order.

- The attacker begins by borrowing 4418 ETH from bZx (Step 1).
- The attacker uses the 4418 borrowed ETH to exchange for sUSD using other DeFi protocols, i.e., Uniswap, Synthetix, and Kyber, in succession (Steps 2–4). Since bZx relies on Uniswap and Kyber for price oracles, which can be affected by a large volume of transactions, the attacker can significantly manipulate the exchange rate of ETH/sUSD in bZx in their favor.
- Subsequently, the attacker borrows an additional 6799 ETH using all 1,099,842 sUSD (Step 5).
- Finally, the attacker repays the initially borrowed 4418 ETH, which was obtained at the outset of the attack (Step 6).

As a result, the attacker realizes a net profit of 2381 ETH (i.e., 2381 = 6799 - 4,418), with only a minimal amount of ETH required to cover gas fees.

Reentrancy. DeFi protocols have also fallen victim to reentrancy attacks. In contrast to traditional reentrancy attacks that target a single smart contract, flash loan reentrancy attacks on DeFi protocols often involve multiple contracts. A reentrancy attack occurs due to untrusted external calls that interrupt atomic transactions, resulting in inconsistent variable states. The core of the flash loan reentrancy attack lies in the exploitation of the inconsistent states within different loan pools before and after a contract's lending (i.e., transferring), allowing them to achieve multiple unregistered loans.

To provide a more concrete illustration of the flash loan reentrancy attack, we present a typical example involving the DeFi protocol Cream.Finance, as depicted in Fig. 6. Here are the key steps of the attack:

- The attacker initiates the attack by borrowing 500 wETH from UniSwapV2 using a flash loan (Step 1).
- Then, the attacker collateralizes the borrowed 500 wETH into Cream.Finance, receiving 24,172 crETH in exchange (Step 2).
- Next, the attacker invokes the “borrow” function within the crAMP pool and exchanges the crETH holdings for 1948 million. However, the transfer of AMP automatically triggers the fallback function within the attacker's contract AMP (Step 3).

¹ dYdX: <https://dydx.exchange>

² bZx: <https://bzx.network>

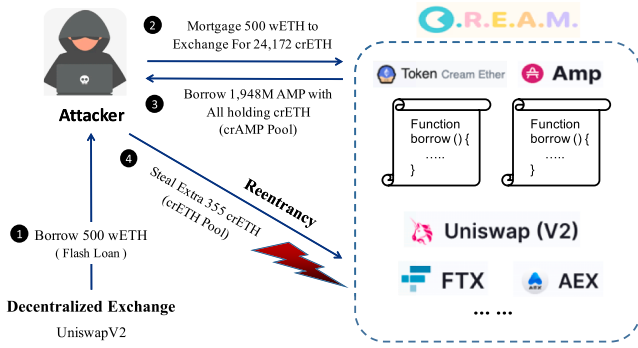


Fig. 6. A reentrancy attack on the DeFi protocol –Cream.Finance.

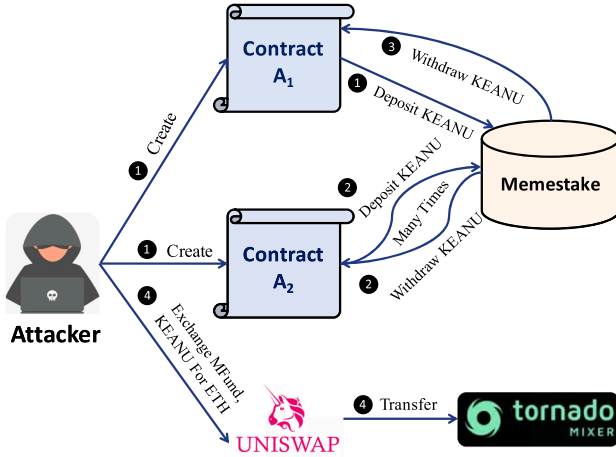


Fig. 7. A real-world deflation token attack on DeFi protocol – Sanshu Inu.

- As the borrowing status of the attacker within the crAMP pool has not been updated, the attacker proceeds to call the “borrow” function within another pool, crETH. This action results in the attacker gaining an additional crETH from the pool (Step 4).

The attacker exploits the reentrancy bug to exploit the inconsistent state of different loan pools of the Cream.Finance protocol, ultimately allowing them to obtain extra crETH tokens beyond what they should have received.

3.2.2. Deflation token attack

Deflation is used to describe the decrease in the value or supply of an asset, often caused by various factors such as over-minting. Token deflation, specifically in the context of decentralized finance, refers to a situation where the total supply of tokens decreases with each token transfer. Deflation tokens are typically designed to deduct a certain percentage of tokens for destruction and redistribution. As a result, the recipient of the tokens may receive an amount less than what the sender originally paid.

Fig. 7 provides a real-world example that illustrates the deflation token attack on the Memestake contract within the DeFi protocol Sanshu Inu³. The attack process can be summarized into the following four steps:

- The attacker starts by creating and initializing two contracts, A₁ and A₂. Contract A₁ serves as an investment contract, depositing a substantial amount of 2049 billion KEANU tokens into the Memestake pool. Contract A₂ is the attack contract, which is used to manipulate the calculation of Memestake’s reward (Step 1).

³ Sanshu Inu: <https://sanshuinufinance.com>

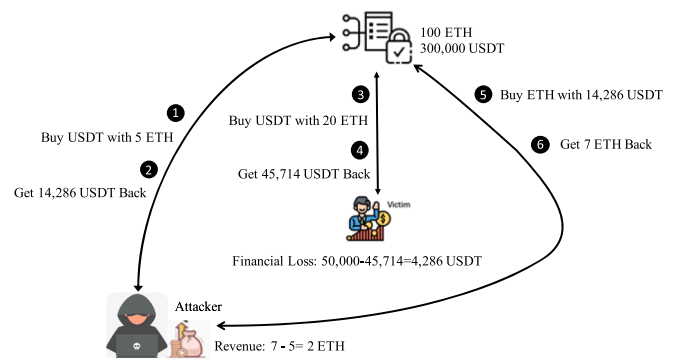


Fig. 8. A simplified example of the sandwich attack on AMM DEXs.

- The attacker then uses A₂ to repeatedly perform a series of deposits and withdrawals of many KEANU tokens to and from Memestake. This action forces Memestake to engage in large-scale KEANU token trading. Since KEANU is a deflation token, each transaction incurs a 2% cost based on the transaction amount. As a result, the actual number of tokens that users deposit into Memestake is smaller than the registered amount in the *user.amount* maintained by Memestake. However, when A₂ initiates the withdrawal operation, Memestake still transfers the recorded amount of tokens to A₂, leading to a continuous reduction of KEANU’s token holdings within the Memestake pool (Step 2).
- The attacker uses A₂ to modify the value of *accMfundPerShare* within the Memestake contract. The actual number of KEANU tokens in Memestake has been significantly reduced to a minimal amount. Consequently, when A₁ proceeds to withdraw the KEANU tokens, it receives a reward in MFund tokens that far exceed the normal value (Step 3).
- Finally, the attacker exchanges the obtained MFund and KEANU tokens for ETH, and subsequently transfers them using a privacy-enhancing tool like Tornado, ultimately resulting in a net profit of 56 ETH (Step 4).

3.2.3. Sandwich attack

A sandwich attack is a predatory trading strategy commonly used in the blockchain and cryptocurrency. A malicious trader orchestrates a sequence of transactions to exploit another user’s trade. The attacker executes two transactions that sandwich the victim’s transaction, with one occurring before and the other after the victim’s trade (Qin, Zhou, & Gervais, 2021b; Wang et al., 2022b). In blockchain systems, transactions are typically ordered in the mempool based on the gas price, and miners prioritize transactions with higher gas prices when creating a block (Zarir, Oliva, Jiang, & Hassan, 2021). Therefore, the attacker can launch a sandwich attack by setting the gas price of the front-running transaction slightly higher than that of the victim’s transaction, and the gas price of the back-running transaction lower than that of the victim’s transaction. Exactly, a sandwich attack exploits the transaction ordering vulnerabilities in the consensus layer.

As depicted in Fig. 8, the attacker employs a front-running strategy to buy USD and a back-running strategy to sell it, effectively sandwiching the victim’s transaction. The victim’s transaction is an attempt to buy USD with 20 ETH. In a scenario without any attackers, the trader would receive 500,000 USD. However, the attacker intervenes in the following manner:

- The attacker initiates a purchase order for USD, acquiring 14,286 USD in exchange for 5 ETH. This action changes the reserves of the liquidity pool. Initially, the ETH/USD ratio was 1/3000, but after the attacker’s first transaction, it shifted to 1/2,721 (Steps 1–2).

- Due to the changes in the liquidity pool, the victim's transaction now allows to obtain only 45,714 USDT for 20 ETH, resulting in a loss of 4286 USDT compared to the initial expectation (Steps ③–④).
- After the victim's transaction, the ETH/USDT ratio of the liquidity pool has further updated to 1/1920. The attacker proceeds to use the remaining 14,286 USDT to execute a sell order for USDT, obtaining 7 ETH in return. This maneuver results in a profit of 2 ETH for the attacker (Steps ⑤–⑥).

3.2.4. Rug pull attack

A rug pull is a deceptive and fraudulent maneuver often seen in DeFi. It involves draining investments from the liquidity pools of DEXs or abruptly abandoning a project, resulting in the loss of investor funds without any prior warning or indication (Scharfman, 2022). The rug pull is a common type of fraud within DeFi. The modus operandi of a rug pull typically unfolds as follows:

- Scammers invest a large number of money in liquidity pools within decentralized exchanges. They then use enticing marketing tactics on social media platforms to lure in unsuspecting investors.
- In response to the appealing advertisements and the promise of lucrative returns, investors deposit their tokens into these liquidity pools.
- When the time is right, the scammers withdraw all the tokens from the liquidity pools. This action leaves investors with nearly empty pools.
- A common variation of the rug pull attack involves manipulating token prices using the attacker's reserves. This manipulation can cause a dramatic drop in the value of the token, effectively rendering it worthless.

It is worth noting that rug pulls in the DeFi area have caused severe consequences, resulting in significant losses for investors. In 2021, these fraudulent schemes accounted for losses exceeding \$2.8 billion, making them the most substantial category of fraud in the DeFi ecosystem, accounting for approximately 37 % of the total fraud revenue for that year (Malwa, 2023b).

3.3. Summary

In light of the complicated and diverse nature of the DeFi ecosystem, we are unable to provide an exhaustive description of every type of DeFi vulnerability and attack. Instead, we have compiled a list of notable DeFi protocols that have fallen victim to attacks over the past three years, as presented in Table 1. For a more comprehensive resource on DeFi related hacks, frauds, and scams, we refer the readers to the (Initiative, 2023; Zhou et al., 2023).

Our analysis of various DeFi attacks has yielded several key observations:

- **Open Source and Interconnected.** DeFi operates on open source principles, which means that its underlying code is available to everyone. While this openness fosters innovation and inclusivity, it also introduces inherent security risks. In addition, DeFi protocols are typically interconnected, with one protocol relying on another for liquidity or functionality. This composability can create a ripple effect, where a vulnerability in one protocol can affect others in the ecosystem, allowing attackers to exploit weaknesses across multiple platforms.
- **External Exploitation.** The complexity and composability of DeFi protocols make them susceptible to external exploitation. The interconnectedness of different protocols can lead to unintended vulnerabilities, where a weakness in one protocol can be exploited through interactions with others. In particular, DeFi protocols are built on blockchain platforms, such as Ethereum, and use smart contracts to automate financial transactions and operations. The complexity of these smart contracts can lead to coding errors or vulnerabilities that attackers can exploit.

- **Flash Loan Exploits.** Flash loans, a feature of some DeFi platforms, allow users to borrow large sums of money without collateral as long as the borrowed funds are returned within a single transaction. This feature can be exploited by attackers to manipulate prices or perform other malicious actions within a short timeframe.
- **Rushed Launch.** Many DeFi applications are hastily launched to take advantage of new market opportunities. In their eagerness to gain a foothold in the emerging financial landscape, some developers may neglect thorough security audits, exposing DeFi protocols to potential security flaws.
- **Lack of Regulation and Experience.** DeFi operates in a largely unregulated environment that can attract malicious actors seeking to exploit regulatory gaps. The absence of oversight and compliance standards can make it easier for attackers to carry out fraudulent activities. Additionally, DeFi is a relatively new field, and many participants, including developers and users, may lack the experience and knowledge required to effectively identify and mitigate security risks.

To sum up, a multitude of factors have converged to create challenges in safeguarding DeFi protocols against a wide array of potential threats and vulnerabilities. To address these issues and enhance security, the DeFi community is diligently engaged in bolstering security practices, conducting meticulous audits, and innovating solutions aimed at fortifying the overall security of DeFi protocols and the broader ecosystem.

4. Security tools for DeFi protocol and smart contract

To ensure the security of participant funds and maintain trustworthiness within DeFi protocols and smart contracts, recent advancements introduce corresponding security solutions, such as vulnerability detection, attack hunting, risk assessment, and automated repair techniques (Table 2). In the following, we present a detailed overview of these techniques within each category.

4.1. Analysis of DeFi vulnerability detection tools

In this subsection, we begin by reviewing existing vulnerability detection tools for DeFi protocols, with the investigation of their design principles. Thereafter, we provide a comprehensive overview and classification of current bug detection tools for smart contracts.

4.1.1. Vulnerability detection for DeFi protocols

Addressing vulnerabilities within DeFi protocols requires a deep understanding of smart contracts, blockchain architecture, and the DeFi ecosystem. Currently, vulnerability detection for DeFi protocols is still in its infancy. Below, we dive into three existing vulnerability detection tools for DeFi protocols, namely PROMUTATOR, DEFTAINTER, and DEFIWARDER.

PROMUTATOR. PROMUTATOR (Wang et al., 2021c) is a scalable security analysis framework designed to identify price oracle bugs in DeFi protocols. PROMUTATOR observes how a price oracle responds to abnormal price data feeds and helps to identify vulnerabilities. Fig. 9 shows the high-level workflow of PROMUTATOR, with detailed steps as follows.

- **Decoding.** PROMUTATOR begins by processing each transaction within the target DeFi protocol. During the decoding phase, it extracts and decodes the relevant information and parameters from these transactions and prepares them for further analysis.
- **Mutation.** In the mutation phase, PROMUTATOR employs a built-in set of mutation rules to simulate various price oracle attacks locally. These mutations involve introducing abnormal or malicious price data feeds to observe how the price oracle reacts.
- **Analysis.** After introducing mutations, PROMUTATOR carefully observes and analyzes the behavior of the price oracle when confronted with abnormal price data feeds. It focuses on identifying any deviations from expected behavior that may indicate vulnerabilities.

Table 1

A summary of well-known DeFi protocols that have suffered from external attacks. Specifically, **Category** means the category of DeFi financial services. **Attack Type** denotes the attack type that a DeFi protocols had endured. **Loss** indicates the corresponding financial losses caused by the attack. **Date** indicates the time when the attack occurred.

#	DeFi Protocols	Category	Attack Type	Loss	Date	Ref.
1	bZx-V1	Lending and Borrowing	Flash Loan Attack	\$9.4 Million	2020.02.17	Kistner (2023b)
2	MakerDAO	Other		\$9 Million	2020.03.12	MakerDAO (2023)
3	UniSwap	Decentralized Exchange		\$0.3 Million	2020.04.18	Team (2023)
4	bZx-V2	Lending and Borrowing		\$8.1 Million	2020.09.15	Kistner (2023a)
5	Eminence Finance	Gaming		\$15 Million	2020.09.29	REKT (2023)
6	Harvest Finance	Derivative		\$33.8 Million	2020.10.26	Finance (2023d)
7	Akropolis	Aggregator		\$2 Million	2020.11.12	Akropolis (2023)
8	Pickle Finance	Yield Aggregator		\$20 Million	2020.11.22	Gottsegen (2023)
9	Warp Finance	Lending and Borrowing		\$7.7 Million	2020.12.17	Finance (2023g)
10	Alpha Homora	Derivative		\$37.5 Million	2021.02.13	Williams (2023b)
11	Spartan	Decentralized Exchange		\$40 Million	2021.05.02	SpartanProtocol (2023)
12	Value DeFi	Derivative		\$28 Million	2021.05.07	REKT (2023a), REKT (2023b), REKT (2023c)
13	Venus	Lending and Borrowing		\$300 Million	2021.05.19	autofarm.network (2023)
14	PancakeBunny	Aggregator		\$47.1 Million	2021.05.20	Finance (2023c), Finance (2023b)
15	Burgerswap	Decentralized Exchange		\$7.2 Million	2021.05.28	Genç (2023)
16	JulSwap	Decentralized Exchange		\$0.7 Million	2021.05.28	JustLiquidity (2023)
17	Belt Finance	Decentralized Exchange		\$50 Million	2021.05.30	Finance (2023a)
18	SushiSwap	Decentralized Exchange		\$0.11 Million	2021.07.21	SlowMist (2023)
19	THORChain	Cross-Chain Bridge		\$5.8 Million	2021.07.22	THORChain (2023)
20	Vee Finance	Lending and Borrowing	Deflation Token Attack	\$13.5 Million	2021.09.21	Finance (2023f)
21	Indexed Finance	Aggregator		\$16 Million	2021.10.14	Finance (2023e)
22	Cream Finance	Lending and Borrowing		\$130 Million	2021.10.27	Thurman (2023)
23	Nerve	Portfolio Management		\$8 Million	2021.11.15	BlockSec (2023a)
24	Beanstalk	Stablecoin		\$182 Million	2022.04.17	Benson (2023a)
25	Saddle Finance	Decentralized Exchange		\$10 Million	2022.04.30	Boyle (2023)
26	Balancer	Portfolio Management		\$0.5 Million	2020.06.28	McDonald (2023)
27	Sanshu Inu	Gaming		\$5 Million	2020.12.28	BlockSec (2023b)
28	Ankr	Other		\$5 Million	2022.12.02	Maiboroda (2023)
29	Lendf.Me	Lending and Borrowing		\$25 Million	2020.04.18	PeckShield (2023b)
30	Origin	Other		\$7 Million	2020.11.17	Liu (2023)
31	Grim Finance	Derivative		\$30 Million	2021.12.19	Benson (2023b)
32	Rari Capital	Lending and Borrowing		\$90 Million	2022.04.30	Lucid (2023); Williams (2023a)
33	FEI protocol	Stablecoin		\$80 Million	2022.04.30	CertiK (2023)
34	Mango	Decentralized Exchange		\$100 Million	2022.10.12	Malwa (2023a)
35	BONDLY	Stablecoin		\$22 Million	2021.07.15	DAO (2023)
36	Bancor	Decentralized Exchange		\$23.5 Million	2020.06.16	Russell (2023)
37	DODO Pool	Decentralized Exchange		\$3.8 Million	2021.03.09	DODO (2023)
38	Uranium Finance	Decentralized Exchange	Sandwich Attack	\$50 Million	2021.04.28	Foundation (2023)
39	ChainSwap	Cross-Chain Bridge		\$4.8 Million	2021.07.11	ChainSwap (2023b), ChainSwap (2023a)
40	Popsicle	Derivative		\$25 Million	2021.08.04	Popsicle.Finance (2023)
41	Poly Network	Cross-Chain Bridge		\$611 Million	2021.08.10	Gagliardi (2023)
42	Compound Finance	Lending and Borrowing		\$160 Million	2021.10.03	Gogo (2023)
43	PAID Network	Decentralized Exchange		\$160 Million	2021.03.05	NETWORK (2023)
44	Ronin	Gaming		\$622 Million	2022.03.23	Hayward (2023)
45	Oypn	Insurance		\$0.37 Million	2020.08.04	PeckShield (2023a)
46	Chainlink	Stablecoin		\$0.34 Million	2020.08.30	Dalton (2023)
47	Cover	Insurance		\$5 Million	2020.12.28	Protocol (2023)
48	yCredit Finance	Decentralized Exchange		\$326 Million	2021.01.02	BlockSec (2023c)
49	Yearn Finance	Aggregator		\$11 Million	2021.02.04	Staff (2023)
50	Furucombo	Aggregator		\$15 Million	2021.02.27	FURUCOMBO (2023)
51	EasyFi	Lending and Borrowing		\$80 Million	2021.04.20	EasyFi (2023)
52	VaultSX	Other		\$13.5 Million	2021.05.16	VaultSX (2023)
53	AnySwap	Decentralized Exchange		\$7.9 Million	2021.07.10	Anyswap (2023)
54	SafeDollar	Stablecoin		\$0.25 Million	2021.07.28	Dollar (2023)
55	BadgerDAO	Other	Compromised Private Key	\$120 Million	2021.12.02	Halborn (2023)
56	Wormhole	Message Passing		\$7.7 Million	2022.02.02	Wormhole (2023)
57	Nomad	Cross-chain Bridge		\$190 Million	2022.08.02	Sam Kessler (2023)

Table 2

Overview of existing security tools that can either detect vulnerabilities, identify attacks, or assess risk against DeFi protocols.

#	Tool	Type	Public Available	Reference
1	PROMUTATOR	Attack Hunting	https://github.com/csienslab/ProMutator	Wang et al. (2021c)
2	DEFITAITNER		https://github.com/kongqp/DeFiTainter	Kong, Chen, Wang, Jiang, and Zheng (2023)
3	DEFIWARDER		https://github.com/Demonhero0/DeFiWarder	Su et al. (2023b)
4	BLOCKEYE		Not Available	Wang et al. (2021a)
5	DEFIRANGER		Not Available	Wu et al. (2021)
6	FLASHOT		Not Available	Cao et al. (2021)
7	DEFISCANNER		Not Available	Wang et al. (2022a)
8	DeFi-Score	Risk Assessment	https://github.com/Consensys/defi-score	Jordan and Jack (2023)
9	DeFi-Safety		Not Available	Rex et al. (2023)

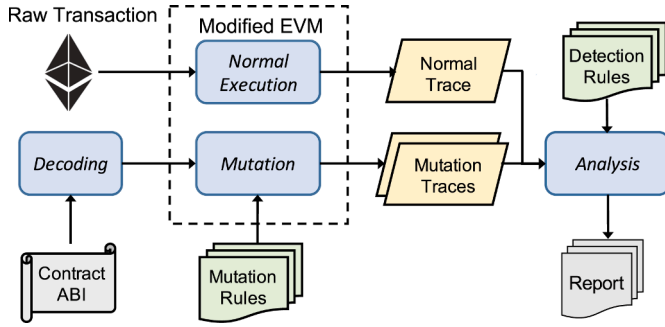


Fig. 9. The high-level workflow of PROMUTATOR.

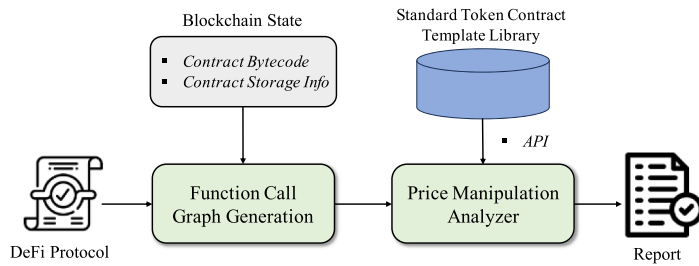


Fig. 10. The overview and workflow of DEFITAINER.

- **Detection Pattern.** PROMUTATOR relies on a set of built-in mutation and detection rules specifically tailored for price oracle vulnerabilities. These rules are designed to detect patterns of behavior that are indicative of vulnerabilities in the price oracle. In addition, PROMUTATOR allows these rules to be extended to cover other types of vulnerabilities by incorporating custom mutation and detection rules.
- **Vulnerability Discovery.** Based on the results of the analysis, PROMUTATOR provides an assessment of whether the target DeFi protocol is vulnerable to price oracle attacks. This assessment is generated by evaluating the detected anomalies against the predefined detection rules.

DEFITAINER. DEFITAINER (Kong et al., 2023) is an inter-contract taint analysis framework for detecting price manipulation vulnerabilities in DeFi protocols. Fig. 10 provides an overview and workflow of DEFITAINER. Given a DeFi protocol, DEFITAINER first utilizes intraprocedural data flow analysis to infer where the call information is stored in the contract storage. Then, it queries the blockchain state to generate the function call graph based on the inferred storage location. In addition, DEFITAINER leverages knowledge of standard contract templates to identify token instant liquidity query interfaces and sensitive transfers, and treats them as taint sources and sinks, respectively. By considering the propagation of taint data between contracts, DEFITAINER can effectively detect price manipulation vulnerabilities.

DEFIWARDER. DEFIWARDER (Su et al., 2023b) aims to trace the on-chain transactions and protect DeFi protocols from token-leaking vulnerabilities. Fig. 11 illustrates the workflow of DEFIWARDER, which consists of three stages. First, DEFIWARDER captures the execution traces of a given DeFi protocol by replaying transactions. Then, it extracts the token transfers within transactions and identifies the roles (user or application) of related addresses, and the relationships between users, thereby generating the token flow of users. Finally, DEFIWARDER merges the token flows and performs anomaly detection to reveal token-leaking vulnerabilities and the corresponding attacker behaviors.

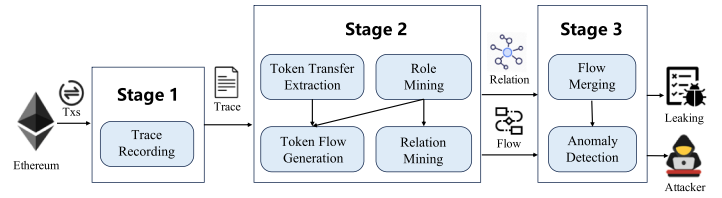


Fig. 11. The overview and workflow of DEFIWARDER.

4.1.2. Vulnerability detection for smart contracts

A plethora of research (Qian, Liu, Yin, & He, 2023b; Qian et al., 2023c) has been undertaken to detecting vulnerabilities in smart contracts using various techniques, such as program analysis, formal verification, fuzz testing, and machine learning. In addition, the hybrid approach, which combines different strategies, has sparked a strong interest in the field of bug detection, as the hybrid analysis expands the ability to discover unknown vulnerabilities. In this context, existing methods can be broadly classified into three categories, i.e., static analysis, dynamic analysis, and hybrid analysis. Table 3 summarizes existing smart contract vulnerability detection tools, while Table 4 lists the types of vulnerabilities they can support.

Static Analysis. Static analysis is a technique for analyzing programs without actually running them. This method scrutinizes code in the absence of real input data, and is adept at identifying potential security vulnerabilities, runtime errors, and logical inconsistencies. The utilization of static analysis techniques has gained significant traction in the area of smart contract vulnerability detection. In practice, static analysis can be broken down into formal verification, symbolic execution, and machine learning.

Formal Verification. Formal verification plays a key role in eliminating ambiguity and ensuring compatibility in smart contracts by translating their concepts, judgments, and logic into a rigorous formal model (Murray & Anisi, 2019). It collaborates with stringent proofs to verify the correctness and security of functions in smart contracts. Commonly used formal verification methods include model checking and deductive verification (Gu, Marinescu, Secleanu, & Lundqvist, 2018).

Symbolic Execution. Symbolic execution methods are a powerful technique for analyzing software programs by using symbolic values as inputs instead of concrete values during execution. When a program branch is encountered, the analyzer captures the associated path constraints and relies on a constraint solver to derive specific input values that can traverse each branch. Symbolic execution has the advantage of exploring multiple paths simultaneously, but it is susceptible to problems such as path explosion.

Machine Learning. In recent years, there has been a notable trend in the field of program security towards leveraging machine learning technologies for vulnerability detection (Lin, Wen, Han, Zhang, & Xiang, 2020). The rapid progress in deep learning has spurred the development of various methods for identifying security vulnerabilities (Chakraborty, Krishna, Ding, & Ray, 2022). In many cases, these methods involve the conversion of smart contracts into intermediate representations, followed by the construction of deep neural networks as detection models. Machine learning's ability to handle complicated data has made it a valuable tool for addressing the complex business logic often found in DeFi protocols. Particularly, we summarize existing machine learning-based approaches that can detect smart contract vulnerabilities in Table 3, hoping to inspire further research in this area.

Dynamic Analysis. Dynamic analysis finds potential security problems in smart contracts by directly executing the contract code, typically via fuzzing test. Fuzzing has proven to be a highly effective technique for discovering software bugs and vulnerabilities over the past few decades. It encompasses a number of methods, including *whitebox*, *blackbox*, and *greybox* (Xu, Moon, Kashyap, Tseng, & Kim, 2019). The core concept

Table 3

The overall classification of existing vulnerability detection tools for smart contracts.

#	Tool	Category	Type	Analysis Level	Public Available	Reference
1	VERISmart	Static Analysis	Formal Verification	Source Code	https://github.com/kupl/VeriSmart-public	So, Lee, Park, Lee, and Oh (2020)
2	VERISOL			Source Code	https://github.com/microsoft/verisol	Wang et al. (2018)
3	ZEUS			Source Code	Not Available	Kalra, Goel, Dhawan, and Sharma (2018)
4	DEFECTCHECKER		Symbolic Execution (SE)	Bytecode	https://github.com/Jiachi-Chen/DefectChecker	Chen et al. (2022)
5	HONEYBADGER			Bytecode	https://github.com/christoftorres/HoneyBadger	Torres, Steichen et al. (2019)
6	MAIAN			Bytecode	https://github.com/MAIAN-tool/MAIAN	Nikolić, Kolluri, Sergey, Saxena, and Hobor (2018)
7	MANTICORE			Bytecode	https://github.com/trailofbits/manticore	Mossberg et al. (2019)
8	MYTHRIL			Bytecode	https://github.com/ConsenSys/mythril	Mueller (2023)
9	OYENTE			Bytecode	https://github.com/melonproject/oyente	Luu, Chu, Olickel, Saxena, and Hobor (2016)
10	OSIRIS			Bytecode	https://github.com/christoftorres/Osiris	Torres, Schütte, and State (2018)
11	SEREUM			Bytecode	Not Available	Rodler, Li, Karame, and Davi (2018)
12	TEETHER			Bytecode	https://github.com/nescio007/teether	Krupp and Rossow (2018)
13	VERX			Source Code	https://github.com/eth-sri/verx-benchmarks	Permenev, Dimitrov, Tsankov, Drachler-Cohen, and Vechev (2020)
14	ETHIR			Bytecode	https://github.com/costa-group/ethIR	Albert, Gordillo, Livshits, Rubio, and Sergey (2018)
15	SAFERSC			Bytecode	https://github.com/wesleyjtann/Safe-SmartContracts	Tann, Han, Gupta, and Ong (2018)
16	RECHECKER	Machine Learning (ML)		Source Code	https://github.com/Messi-Q/ReChecker	Qian, Liu, He, Zimmermann, and Wang (2020)
17	S-GRAM			Source Code	https://github.com/njaliu/sgram-artifact	Liu, Liu, Zhao, Jiang, and Sun (2018b)
18	TMP			Source Code	https://github.com/Messi-Q/GNNSCVulDetector	Zhuang et al. (2020), Liu et al. (2023b)
19	CONTRACTWARD			Bytecode	Not Available	Wang et al. (2020b)
20	DEESCVHUNTER		Other Program Analysis	Source Code	Not Available	Yu, Zhao, Hou, Ying, and Wu (2021)
21	CODENET			Source Code	Not Available	Hwang, Choi, Shin, and Choi (2022)
22	DL-MDF			Source Code	Not Available	Deng et al. (2023)
23	SMARTCHECK			Source Code	https://github.com/smartdec/smartcheck	Tikhomirov et al. (2018)
24	SECURIFY			Bytecode	https://github.com/eth-sri/securify2	Tsankov et al. (2018), Tsankov et al. (2023)
25	SLITHER			Source Code	https://github.com/cryptic/slither	Feist et al. (2019)
26	VANDAL			Bytecode	https://github.com/usyd-blockchain/vandal	Brent et al. (2018)
27	CONTRACTFUZZER			Bytecode	https://github.com/gongbell/ContractFuzzer	Jiang et al. (2018)
28	CONTRAMASTER	Dynamic Analysis	Fuzzing Test	Source Code	https://github.com/ntu-SRSLab/vultron	Wang et al. (2020a), Wang, Li, Lin, Ma, and Liu (2019)
29	REGURAD			Source Code	Not Available	Liu et al. (2018a)
30	HARVEY			Source Code	Not Available	Wüstholtz and Christakis (2020a), Wüstholtz and Christakis (2020b)
31	sFUZZ			Bytecode	https://github.com/duytai/sFuzz	Nguyen, Pham, Sun, Lin, and Minh (2020)
32	IR-FUZZ	Hybrid Analysis	Fuzzing + SE + ML	Source Code	https://github.com/Messi-Q/IR-Fuzz	Liu et al. (2023c)
33	ITYFUZZ			Source Code	https://github.com/fuzzland/ityfuzz	Shou, Tan, and Koushik (2023)
34	ILF			Source Code	https://github.com/eth-sri/ilf	He, Balunović, Ambroladze, Tsankov, and Vechev (2019)
35	SMARTIAN		Fuzzing + SE	Bytecode	https://github.com/SoftSec-KAIST/Smartian	Choi et al. (2021)
36	CONFUZZIUS			Bytecode	https://github.com/christoftorres/Confuzzius	Torres, Iannillo, Gervais, and State (2021)
37	xFUZZ		Fuzzing + ML	Source Code	https://github.com/ToolmanInside/xfuzz_tool	Xue et al. (2022)
38	RLF			Source Code	https://github.com/Demonhero0/rlf	Su, Dai, Zhao, Zheng, and Luo (2023a)

Table 4

A summary of bug types supported by each tool. For each type: AC is shot for access control; AF implies assertion failure; BD indicates block dependency (block.timestamp, block.number, block.blockhash, etc); DS refers to denial of service; DD denotes dangerous delegatecall; FE represents freezing Ether; GS means gasless send; LE is leaking Ether; RE stands for reentrancy; IO denotes integer over-/under- flow; SC implies suicidal contract; SA refers to short addresses; TO is transaction order dependency; TX means transaction origin use; UC indicates unchecked low-level call; UE is unhandled exception. For more details of various smart contract vulnerabilities, we refer interested readers to [Atzei et al. \(2017\)](#), [Durieux et al. \(2020\)](#), [Ren et al. \(2021\)](#), [Rameder et al. \(2022\)](#), [Vidal et al. \(2023\)](#).

Tool	Vulnerability Type															
	AC	AF	BD	DS	DD	FE	GS	LE	RE	IO	SC	SA	TO	TX	UC	UE
CODENET (Hwang et al., 2022)	X	X	✓	X	X	X	X	X	✓	X	X	X	X	✓	✓	X
CONFUZZIUS (Torres et al., 2021)	X	✓	✓	X	✓	✓	X	✓	✓	✓	✓	X	✓	X	X	✓
CONTRACTFUZZER (Jiang et al., 2018)	✓	X	✓	X	✓	✓	✓	X	✓	X	X	X	X	X	X	✓
CONTRAMASTER (Wang et al., 2019, 2020a)	X	X	X	X	X	X	✓	X	✓	✓	X	X	X	X	X	✓
CONTRACTWARD (Wang et al., 2020b)	X	X	✓	X	X	X	X	X	✓	✓	X	X	✓	X	X	X
DEESCVHUNTER (Yu et al., 2021)	X	X	✓	X	X	X	X	X	✓	X	X	X	X	X	X	X
DEFECTCHECKER (Chen et al., 2022)	X	X	✓	✓	X	✓	X	✓	✓	X	X	X	✓	X	✓	X
DL-MDF (Deng et al., 2023)	X	X	✓	X	X	✓	X	X	✓	✓	X	X	X	X	X	X
ETHIR (Albert et al., 2018)	✓	✓	✓	X	✓	X	X	X	✓	X	X	X	✓	X	X	X
HARVEY (Wüstholtz & Christakis, 2020a,b)	X	✓	X	X	X	X	X	X	✓	✓	X	X	X	X	✓	✓
HONEYBADGER (Torres et al., 2019)	X	X	X	X	X	✓	X	✓	X	✓	X	X	X	X	X	X
ILF (He et al., 2019)	✓	X	✓	X	✓	✓	X	✓	✓	X	✓	X	X	X	X	✓
IR-FUZZ (Liu et al., 2023c)	✓	X	✓	X	✓	✓	✓	X	✓	✓	✓	X	X	✓	✓	✓
ITYFUZZ (Shou et al., 2023)	✓	X	X	✓	X	X	X	X	✓	X	X	X	X	X	✓	X
MAIAN (Nikolić et al., 2018)	✓	X	X	X	X	✓	X	✓	X	X	✓	X	X	X	X	X
MANTICORE (Mossberg et al., 2019)	✓	✓	✓	X	✓	X	X	✓	✓	✓	✓	X	X	✓	✓	✓
MYTHRIL (Mueller, 2023)	✓	✓	✓	X	✓	X	X	✓	✓	✓	✓	X	X	✓	✓	✓
OYENTE (Luu et al., 2016)	✓	✓	✓	✓	X	X	X	X	✓	X	X	X	✓	X	X	X
OSIRIS (Torres et al., 2018)	X	✓	✓	✓	X	X	X	X	✓	✓	X	X	X	X	X	X
RECHECKER (Qian et al., 2020)	X	X	X	X	X	X	X	X	✓	X	X	X	X	X	X	X
REGURAD (Liu et al., 2018a)	X	X	X	X	X	X	X	X	✓	X	X	X	X	X	X	X
RLF (Su et al., 2023a)	X	X	✓	X	✓	✓	X	✓	X	X	✓	X	X	X	X	✓
SAFERSC (Tann et al., 2018)	X	X	X	X	X	✓	X	✓	X	X	✓	X	X	X	X	X
SECURIFY (Tsankov et al., 2018, 2023)	✓	X	✓	X	✓	✓	X	✓	✓	✓	✓	X	✓	✓	✓	✓
SEREUM (Rodler et al., 2018)	X	X	X	X	X	X	X	X	✓	X	X	X	X	X	X	X
S-GRAM (Liu et al., 2018b)	X	✓	✓	X	X	✓	X	X	✓	✓	X	X	X	X	X	X
sFUZZ (Nguyen et al., 2020)	✓	X	✓	X	✓	✓	✓	X	✓	✓	X	X	X	X	X	✓
SMARTCHECK (Tikhomirov et al., 2018)	✓	X	✓	✓	X	✓	X	X	✓	✓	X	X	X	✓	X	✓
SMARTIAN (Choi et al., 2021)	X	✓	✓	X	✓	✓	X	✓	✓	✓	✓	X	X	✓	✓	✓
SLITHER (Feist et al., 2019)	✓	X	✓	✓	✓	✓	X	✓	✓	X	✓	X	X	✓	✓	✓
TEETHER (Krupp & Rossow, 2018)	X	X	X	X	✓	X	X	✓	X	X	✓	X	X	X	X	X
TMP (Liu et al., 2023b; Zhuang et al., 2020)	X	X	✓	X	X	X	X	X	✓	X	X	X	X	X	X	X
VANDAL (Brent et al., 2018)	X	X	X	X	X	X	X	✓	✓	X	✓	X	X	✓	✓	✓
VERISMAST (So et al., 2020)	X	X	X	X	X	X	X	✓	X	✓	X	X	X	X	X	X
VERISOL (Wang et al., 2018)	X	✓	X	X	X	X	X	X	X	X	X	X	X	X	X	X
VERX (Permenev et al., 2020)	✓	✓	X	X	X	X	X	X	X	✓	X	X	X	X	X	X
xFUZZ (Xue et al., 2022)	X	X	✓	X	✓	X	X	X	✓	X	X	X	X	✓	X	X
ZEUS (Kalra et al., 2018)	X	X	✓	X	X	X	X	X	✓	✓	X	X	✓	✓	✓	✓

behind fuzzing techniques is to inundate a program under test with a variety of test inputs or test cases, with the goal of identifying vulnerabilities by monitoring for abnormal results or exceptions that may occur during execution ([Jiang et al., 2018](#)).

Hybrid Analysis. Smart contract bug detection techniques have evolved into a hybrid analysis, which synergistically combines static and dynamic analysis. This approach has demonstrated promising results in vulnerability detection. Particularly, static analysis methods can help the fuzzer to generate better test cases and detect more bugs. For example, ILF ([He et al., 2019](#)) is an end-to-end system that learns an effective and fast fuzzer from symbolic execution by framing the learning task within the imitation learning framework. CONFUZZIUS ([Torres et al., 2021](#)) introduces a hybrid smart contract fuzzer that employs symbolic execution to explore uncovered branch conditions and dynamically creates meaningful transaction sequences by exploiting data dependencies.

4.2. Analysis of DeFi attack hunting tools

In the field of DeFi attack detection, it is crucial not only to identify vulnerabilities but also to comprehend the complex logical semantics of DeFi protocols. Traditional vulnerability detection methods for smart contracts often fall short in capturing such semantics. To address this gap, recent DeFi attack detection techniques simulate potential attack

scenarios to uncover malicious behavior ([Wang et al., 2021a,c; Wu et al., 2021](#)). These approaches monitor contract transactions during the simulation and leverage high-level semantic patterns of DeFi protocols to identify attack traces. The following summarizes the overall design and workflow of representative DeFi attack hunting strategies:

- **Attack Simulation.** Tools initiate simulated attack scenarios within DeFi protocols by crafting and executing transactions that emulate common actions, such as token swaps, liquidity provision, and lending or borrowing.
- **Transaction Monitoring.** During simulation, tools closely monitor transaction-level events, tracking asset flows, contract interactions, and anomalies that may indicate exploitative behavior.
- **Semantic Understanding.** A core component is recovering high-level protocol semantics via code analysis, which involves identifying key functions and understanding the flow of assets and data across contract components.
- **Pattern Matching.** By combining semantic insights with known attack signatures, tools apply pattern recognition to detect suspicious transaction sequences indicative of DeFi exploits.
- **Alert and Report.** Upon detecting behaviors that resemble attacks, the tools generate alerts or reports that detail the anomaly, the affected contracts, and the associated risks.

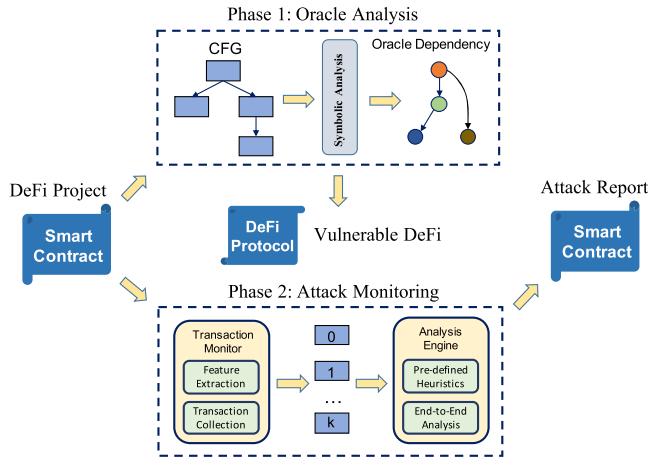


Fig. 12. The overall workflow of BLOCKEYE, which consists of oracle analysis and attack monitoring.

We characterize existing DeFi attack detection methodologies. By dissecting their design principles and operational workflows, our goal is to contribute to the advancement of research in the field of DeFi attack detection, hoping to push the boundaries of knowledge in DeFi security. Below, we elaborate on the specific design and workflow of existing attack hunting tools.

BLOCKEYE. BLOCKEYE (Wang et al., 2021a) is a real-time attack detection system tailored for DeFi protocols. BLOCKEYE is underpinned by two key insights: symbolic oracle analysis and pattern-based runtime transaction validation. To provide a clearer picture of BLOCKEYE's functionality, we describe its workflow in Fig. 12. BLOCKEYE operates in two distinct phases.

- **Phase 1: Symbolic Analysis.** BLOCKEYE performs a symbolic analysis of smart contracts within a given DeFi protocol. The goal of this phase is to establish inter-DeFi oracle dependency, effectively understanding how the data provided by one DeFi protocol's oracle affects the operations of others. BLOCKEYE identifies state updates that are dependent on the oracle, marking these DeFi protocols as potentially vulnerable to attack.
- **Phase 2: Runtime Monitoring.** BLOCKEYE deploys a runtime monitor specifically for the DeFi protocols identified as vulnerable in phase 1. BLOCKEYE employs a transaction monitor to track and collect transactions related to the identified vulnerable DeFi protocols. It does so by extracting key transaction characteristics. The collected transactions undergo an end-to-end analysis based on predefined heuristics. For example, BLOCKEYE looks for abnormal patterns, such as an unusually large profit made in a short period of time. If an abnormal sequence of transactions is detected, BLOCKEYE raises a flag to indicate a possible attack.

DEFIRANGER. DEFIRANGER (Wu et al., 2021) is a specialized tool designed to identify and detect price manipulation attacks within DeFi protocols. DEFIRANGER employs the high-level DeFi semantics to detect price manipulation attacks using predefined patterns. Let us look at the workflow of DEFIRANGER, as illustrated in Fig. 13.

- **Data Collection.** DEFIRANGER initiates its operation by collecting raw Ethereum transactions that are relevant to the DeFi protocols. These transactions represent the raw data of the Ethereum blockchain.
- **Cash Flow Tree Construction.** Once the raw transaction data is collected, DEFIRANGER proceeds to construct the cash flow tree (CFT). The CFT serves as a crucial component in the workflow, allowing

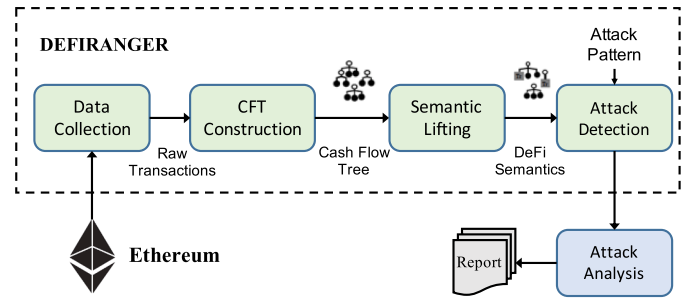


Fig. 13. The general workflow of DEFIRANGER.

the conversion of raw transactions into a well-structured representation, focusing on token transfers and the flow of assets in the DeFi ecosystem.

- **Semantic Recovery.** With the CFT in place, DEFIRANGER utilizes a lifting algorithm to recover the high-level semantics of the DeFi protocols. This phase involves translating the low-level, technical transaction data into meaningful and interpretable DeFi-related semantics.
- **Price Manipulation Attack Detection.** Armed with the recovered DeFi semantics, DEFIRANGER proceeds to detect price manipulation attacks within the DeFi protocols. It accomplishes this by comparing the recovered semantics to predefined attack patterns or rules that are indicative of price manipulation behavior.

FLASHOT. FLASHOT (Cao et al., 2021) is a specialized framework with the unique ability to provide a detailed and precise visualization of the intricate processes involved in a flash loan attack. Beyond visualization, FLASHOT performs an in-depth analysis of typical flash loan attacks, including scenarios such as pumping and arbitrage. This analysis includes creating a more accurate model and solution for the optimization problem at hand. Additionally, FLASHOT dives into providing economic explanations for the malicious behavior exhibited by attackers during such flash loan attacks. In essence, FLASHOT serves as a valuable tool for dissecting and understanding the inner workings of flash loan attacks, providing both visual clarity and detailed analysis to enhance our understanding of these complex financial maneuvers within the DeFi ecosystem.

DEFISCANNER. DEFISCANNER (Wang et al., 2022a) is a deep learning-based attack detection system designed to identify attacks against DeFi protocols. It captures various types of features from external transactions to detect DeFi attacks effectively. Specifically, DEFISCANNER employs a global model, a local model, and a fusion model to characterize DeFi attacks. The overall architecture is illustrated in Fig. 14. First, it automatically normalizes unstructured emitted events. Then, it enriches the transaction-related features of the normalized emitted events using the global model and extracts semantic features of the emitted events with the local model. Finally, DEFISCANNER combines the transaction-related features and the semantic features of the emitted events with the fusion model for DeFi attack detection.

Unlike passive vulnerability scanning, attack hunting in DeFi is an active, simulation-driven process. Instead of merely checking for static code weaknesses, attack hunting proactively explores the operational space of the protocol to identify emerging attack vectors. It requires deep protocol comprehension and dynamic analysis capabilities, as the attacker's behavior is often encoded in multi-step financial interactions. Effective attack hunting tools blend symbolic execution, on-chain data analysis, and domain-specific heuristics to simulate realistic adversarial strategies, making them powerful complements to traditional security analysis techniques.

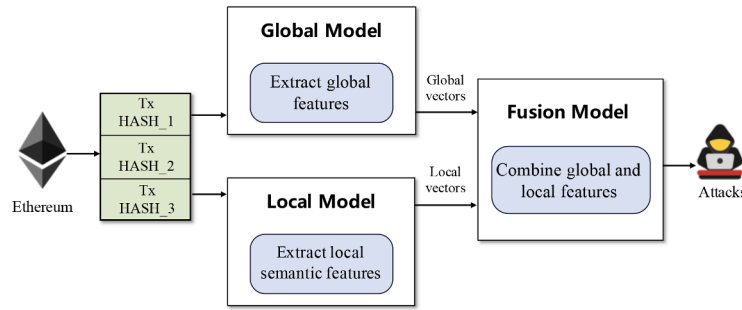


Fig. 14. The overall architecture of DEFISCANNER.

Table 5

Overview of existing security tools for repairing vulnerabilities in DeFi protocols and smart contracts. **Number** represents that the number of vulnerability types can be repaired by each tool.

#	Tool	Analysis Level	Number	Public Available	Reference
1	DEFINERY ^a	Source Code	4	https://sites.google.com/view/ase2022-definery	Tolmach, Li, and Lin (2022)
2	SCREPAIR	Source Code	4	https://github.com/xiaoly8/SCRepair	Yu, Al-Bataineh, Lo, and Roychoudhury (2020)
3	SMARTSHIELD	Bytecode	3	Not Available	Zhang et al. (2020)
4	SGUARD	Source Code	4	https://github.com/duytai/sGuard	Nguyen, Pham, and Sun (2021b)
5	EVMPATCH	Bytecode	2	https://github.com/uni-due-syssec/evmpatch-developer-study	Rodler, Li, Karame, and Davi (2021)
6	AROC	Source Code	4	Not Available	Jin et al. (2021)
7	HCC	Source Code	2	Not Available	Giesen, Andreina, Rodler, Karame, and Davi (2022)
8	ELYSIUM	Bytecode	7	https://github.com/christoftorres/Elysium	Ferreira Torres, Jonker, and State (2022)

^a DEFINERY is built on SCREPAIR. DEFINERY enables property-based automated repair while providing formal correctness guarantees.

4.3. Analysis of DeFi Pprotocol risk assessment tools

Providing a credible risk assessment is also crucial for enabling developers and investors to proactively identify potential risks within a DeFi protocol. Such assessments not only empower crypto participants to make informed decisions but also serve as incentives for DeFi protocols to prioritize and enhance their overall security and trustworthiness. The goal of developers offering risk assessment tools is to foster the safe and sustainable growth of the DeFi ecosystem, making it more secure for users. Two prominent rating tools, DeFi-Score and DeFi-Safety, are noteworthy in this regard.

DeFi-SCORE. DeFi-Score (Jordan & Jack, 2023) serves as a comprehensive framework for evaluating risk in permissionless lending platforms. It provides a quantifiable metric to measure the risk associated with DeFi protocols. This measurement takes into account several factors, including smart contract risk, collateralization, and liquidity.

DeFi-SAFETY. DeFi-Safety (Rex et al., 2023) operates as an independent rating organization dedicated to the evaluation of DeFi protocols. Its primary focus is to provide an overall safety score for DeFi projects based on specific criteria. These criteria often include levels of transparency and adherence to best practices.

Both of these assessment tools play an important role in the advancement of DeFi security. Not only do they provide valuable insights for users and investors, but they also encourage DeFi projects to adhere to high standards of security and transparency. The collaborative effort to share tools, standards, and development patterns contributes to the broader goal of making the DeFi ecosystem more secure and trustworthy for all participants.

4.4. Analysis of automated repair tools

While existing tools are capable of detecting vulnerabilities and identifying the corresponding vulnerable code regions in smart contracts, they lack the ability to automatically repair them. Automated smart contract repair has thus emerged as a promising direction. However, the immutable nature of blockchain renders traditional software repair techniques inapplicable, as once deployed, smart contracts cannot be modified. This introduces unique challenges for automated patch generation in the context of decentralized financial systems. In response, re-

cent studies have proposed automated repair approaches tailored to the characteristics of smart contracts and DeFi protocols. These approaches include template-based patching, vulnerability-specific fixes, and symbolic execution-guided repair strategies. Despite this progress, the literature lacks a comprehensive survey or evaluation of these tools in terms of repair soundness, security guarantees, and functional correctness. To bridge this gap, we conducted a systematic review of eight existing automated repair tools for smart contracts, as summarized in Table 5.

DEFINERY. Tolmach et al. (2022) design an approach, DEFINERY, that enables property-based automated repair for DeFi protocols while ensuring formal correctness guarantees. DEFINERY focuses on repairing functional specification violations within DeFi smart contracts and consists of two main modules:

- The semantic analysis module symbolically executes a DeFi smart contract with respect to a specific property and a sequence of functions that result in a violation of that property. This phase is responsible for identifying the problems and violations within the behavior of the contract.
- The patch generation module of DEFINERY is designed to provide an updated version of the DeFi smart contract that fixes the identified property violations. Notably, this module builds upon the foundation of SCREPAIR but extends its capabilities. It combines the semantic analysis module to perform the patching process and incorporates heuristics to select changes that are most likely to effectively repair the identified problem.

By integrating these two modules, DEFINERY provides a comprehensive solution for addressing functional specification violations within DeFi smart contracts. The tool leverages symbolic execution, property analysis, and patch generation to ensure that repaired contracts not only satisfy formal correctness guarantees, but also meet the intended functional specifications, thereby increasing the security and reliability of DeFi protocols.

SCREPAIR. Yu et al. (2020) propose SCREPAIR, an automated smart contract repair tool that employs a genetic programming search method. SCREPAIR's approach involves parallel genetic repair, which entails partitioning the extensive search space of candidate patches into smaller,

mutually exclusive search space that can be processed independently. SCREPAIR bears inherent limitations:

- One limitation of SCREPAIR is that it may generate incomplete patches. These partial solutions can sometimes be ineffective in fully addressing vulnerabilities or issues within the smart contract.
- In some cases, the patches selected by SCREPAIR may inadvertently break the functionality of the original smart contract. Repair tools must strike a delicate balance between fixing vulnerabilities and preserving the intended behavior of the contract.
- SCREPAIR is designed to repair on-chain smart contracts, but is unable to address vulnerabilities or issues in off-chain contracts. Off-chain components of DeFi protocols may still remain vulnerable.

SMARTSHIELD. Zhang et al. (2020) have introduced SMARTSHIELD, a bytecode rectification system designed to automatically address three specific types of bugs in smart contracts. The overall workflow of SMARTSHIELD encompasses three fundamental steps:

- In the first step, SMARTSHIELD extracts the abstract syntax tree and captures the unfixed bytecode of a smart contract. This process allows the system to capture bytecode-level semantic information, which is essential for understanding the behavior of the contract.
- The second step involves the generation of patches to rectify issues related to insecure control flows and data operations within the smart contract. These patches are generated based on the semantic information obtained in the previous step. The goal is to design corrective measures that effectively address the identified vulnerabilities.
- Once the patches have been applied, SMARTSHIELD outputs the patched bytecode, which represents the corrected version of the smart contract. It also generates a bug repair report, providing insights into the repairs during the process.

SGUARD. Nguyen et al. (2021b) presents an automatic fixing approach aimed at eliminating four specific types of vulnerabilities within smart contracts. The approach is carried out in two distinct phases:

- In the first phase, SGUARD collects a finite set of symbolic execution traces from the smart contract. It then performs static analysis on these traces to identify potential vulnerabilities. This process allows SGUARD to detect and understand the vulnerabilities in the contract.
- In the second phase, SGUARD applies specific fixing patterns tailored for each type of bug found in the contract. These patches are implemented at the source code level with the intention of fixing the identified issues.

However, one limitation of SGUARD is its reliance on source code rewriting without performing a more comprehensive analysis of the interactions between different functions and the global dependencies of memory variables. This limitation can lead to unintended consequences, such as breaking the functionality of the original contract.

EVMPATCH. Rodler et al. (2021) propose an automated repair framework designed to address issues within faulty smart contracts via a two-step process:

- EVMPATCH first employs a bytecode rewriting engine that operates based on predefined patch rules. It focuses on repairing individual basic blocks within the contract's bytecode that contain bugs. Once a basic block is fixed, the modified block is appended to the end of the contract's bytecode.
- Following the repair process, EVMPATCH conducts a verification step. It involves replaying selected historical transactions from the original contract on a local Ethereum client. This replay is used to assess the correctness of the fixed contract. EVMPATCH verifies the integrity of the contract by observing whether attack transactions are successfully blocked and whether other transactions proceed as expected.

However, one limitation of EVMPATCH is its suitability for vulnerabilities confined within a single bytecode basic block. The framework faces challenges when dealing with vulnerabilities that span multiple separated basic blocks, indicating difficulty in addressing vulnerabilities that involve complex interactions across different parts of a smart contract bytecode.

AROC. The smart contract repair tool AROC, proposed by Jin et al. (2021), provides automated vulnerability fixing for on-chain contracts without the need to update the contract code. AROC comprises three primary components:

- Given vulnerable contract source code and bug types, AROC captures the variable dependencies, path constraints, and contract metadata to synthesize patches.
- The patch generation module utilizes predefined repair templates to produce patches that are tailored for the specific bugs in the contract.
- AROC integrates with an enhanced version of the EVM. This enhanced EVM binds the generated patches to fix the vulnerabilities within the target contract and to block any potentially malicious transactions.

However, one limitation of AROC is that it currently only works with its own enhanced EVM, and its binding capabilities are not yet compatible with the official Ethereum network. This constraint may limit its practical applicability in the broader Ethereum ecosystem until it can be accepted and integrated into the official Ethereum platform.

HCC. Giesen et al. (2022) develop a smart contract compiler, HCC, designed to automatically insert security hardening checks into contract source code to enhance security. The process in HCC involves several key steps:

- HCC uses a code property graph (CPG) to model the control and data flows within the contract. This CPG is essential for identifying potential vulnerabilities or bugs in the code.
- Based on the CPG, HCC identifies potential vulnerabilities within the contract source code. This step involves analyzing the control and data flows to pinpoint areas where security improvements are needed.
- HCC addresses the identified vulnerabilities by applying hardening patches to the contract code. These patches are expressed as graph patterns that serve as templates for making CPG-level changes to mitigate the identified vulnerabilities.

However, HCC faces challenges when trying to fix vulnerabilities in complex DeFi protocols. This limitation arises because HCC struggles to recover complete code property graphs from intertwined DeFi protocols. Complex interactions and dependencies make it difficult to create accurate and comprehensive CPGs, which are crucial for the bug discovery and patching process.

ELYSIUM. Ferreira Torres et al. (2022) propose ELYSIUM, a scalable tool for automated repair of smart contracts at the bytecode level. The tool follows a comprehensive workflow with four phases, which are illustrated in Fig. 15.

- ELYSIUM is first responsible for detecting and localizing bugs within the smart contract bytecode. It identifies areas where there are vulnerabilities.
- After detecting bugs, ELYSIUM extracts the control flow graph (CFG) of the bytecode. It also derives the necessary context information associated with the CFG. This context information is critical to understanding the relationships and dependencies within the bytecode of the contract.
- Based on the previously inferred context information, ELYSIUM generates patches for the identified bugs. These patches are created by

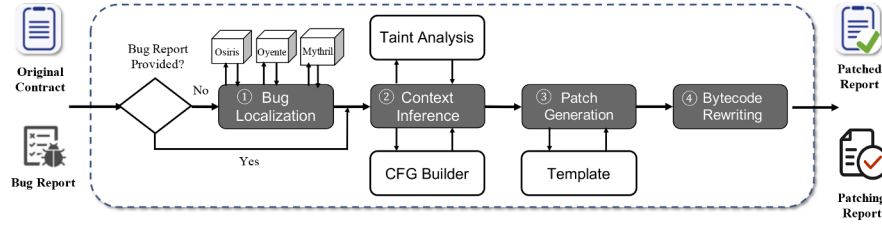


Fig. 15. The overall workflow and architecture of ELYSIUM.

inserting the inferred context information into predefined patch templates. The goal of this step is to provide solutions to the identified vulnerabilities.

- The final phase of ELYSIUM involves bytecode rewriting. In this step, the generated patches are injected into the CFG, effectively changing the control and data flow within the bytecode. The CFG is then converted back to bytecode with the applied patches.

This workflow allows ELYSIUM to automatically locate, analyze, and repair smart contract vulnerabilities at the bytecode level. This approach is designed to further improve the security and reliability of smart contracts.

In summary, the inherent characteristics of smart contracts and DeFi systems, such as blockchain immutability, tightly coupled contract interactions, and complex asset management semantics, make automatic repair significantly more challenging than in traditional software. Current tools still face major limitations in ensuring patch correctness, preserving protocol logic, and analyzing inter-contract dependencies. Moreover, the absence of standardized evaluation criteria and reproducible benchmark environments further hinders both practical deployment and academic progress. As such, despite their promise, automated repair solutions for DeFi protocols remain an open and technically demanding research problem.

4.5. Summary

4.5.1. Insight for DeFi vulnerability and attack detection tools

After a detailed examination of existing bug detection tools for DeFi protocols and smart contracts, we have the following observations:

- **Composability Challenge of DeFi Protocol.** DeFi protocols introduce a significant analytical challenge due to their composability. Unlike traditional smart contracts, DeFi protocols are typically composed of multiple interacting smart contracts. Detecting vulnerabilities in this context can be particularly challenging because these contracts exhibit complex and interrelated behaviors.
- **Focus on Traditional Smart Contracts.** Most state-of-the-art vulnerability detection methods are specifically tailored for traditional smart contracts or single contract scenarios. These tools may not adequately address the unique complexities presented by DeFi protocols.
- **Urgent Need for Scalable Tools.** The dynamic and ever-changing nature of DeFi highlights the urgent need to design and implement scalable bug detection tools. These tools should be able to identify a wide range of vulnerabilities and attacks within the DeFi protocols, even as the ecosystem continues to evolve.

Addressing these problems is paramount to ensuring the security and stability of the DeFi ecosystem. Developing advanced, adaptable, and scalable tools that can effectively analyze the composability and complexity of DeFi protocols is critical to protecting the interests and assets of DeFi users and stakeholders.

4.5.2. Insight for DeFi automated repair tools

Upon investigating existing automated repair tools for DeFi protocols and smart contracts, we have the following observations:

- DeFi protocols often involve intricate interactions between multiple smart contracts, which increases the difficulty of identifying and repairing vulnerabilities. This complexity presents unique challenges that current repair techniques struggle to address effectively.
- Many existing automated smart contract repair techniques are designed to address specific types of vulnerabilities. As a result, they may not provide comprehensive coverage for the wide range of vulnerabilities that can exist in complex smart contracts such as DeFi protocols.
- Ensuring the correctness of patches and validating that the functionality of the patched contract is equivalent to the original contract poses a significant challenge.

Efforts to enhance the capabilities of automated repair tools, expand their vulnerability coverage, and develop robust verification mechanisms are essential for improving the automated repair of DeFi protocols and smart contracts.

5. Performance evaluation

In this section, we aim to evaluate the performance of existing security tools in handling DeFi protocols. We seek to answer the following questions.

- **RQ1:** What is the performance of existing DeFi security tools in detecting DeFi vulnerabilities and attacks?
- **RQ2:** Can traditional smart contract vulnerability detection tools work within complex DeFi protocols? How do they perform in detecting vulnerabilities against DeFi protocols?
- **RQ3:** How efficient are DeFi security tools running on DeFi protocols? What is the runtime of traditional smart contract analysis tools against DeFi protocols?

The following parts of this section present the setup and procedure of the experiments. First, we construct a benchmark dataset of DeFi protocols and select candidate security tools for the experiments. Then, we present the experimental setup, including metrics that are commonly used to evaluate the tools and the execution environment. Next, in a unified execution environment, each tool analyzes DeFi protocols and generates results. We count the total number of reported bugs, manually confirm the authenticity, and calculate the value of relevant metrics, followed by analyzing the experimental results. Finally, we also provide a validation and limitation analysis of the experimental results.

Experimental Settings. We set a uniform execution environment for each tool, including the unified platform and consistency of runtime parameters. Considering that each tool has an arbitrary number of self-defined settings, we choose the general settings that they all support to perform experiments. All experiments are performed on the same computer equipped with an Intel Core i9 CPU at 3.3GHz, an NVIDIA RTX 2080Ti GPU, and 64GB RAM.

5.1. Dataset construction

We construct and release a benchmark dataset, which consists of real-world DeFi protocols and concerns six types of DeFi attacks,

Table 6

Statistical analysis of the DeFi protocol dataset. Category represents the classification of DeFi financial services. FL indicates the flash loan attack; DT refers to deflation token attack; SW denotes DeFi sandwich attack; RP means rug pull attack; Other Attack designates uncommon types of attacks; Normal means a DeFi protocol that has no reported attack.

Category	FL	DT	SW	RP	Other Attack	Normal	Total
Lending and Borrowing	9	2	3	–	1	3	18
Decentralized Exchange	12	1	3	2	3	19	40
Portfolio Management	3	–	1	–	1	2	7
Derivative	6	–	2	1	5	11	25
Aggregator	1	1	–	–	1	3	6
Other	–	–	–	–	–	3	3
Total	31	4	9	3	11	41	99

namely, *flash loan (pump and arbitrage, price manipulation, reentrancy) attack, deflation token attack, sandwich attack, and rug pull attack*. According to the reported DeFi attack incidents, we manually label the dataset, i.e., whether a DeFi protocol suffers from an external attack or has a certain type of DeFi vulnerability. Our annotations extensively facilitate the evaluation of DeFi protocol security tools.

5.1.1. Dataset collection and processing

We create the dataset by collecting DeFi protocols from three different sources: (i) Ethereum official platform (more than 98 %), (ii) GitHub repositories, and (iii) blog posts that analyze DeFi protocols. Note that all the DeFi protocols are collected from trusted entities in the field. We also ensure the traceability of each DeFi protocol by providing the address, from which they are extracted as much as possible. In particular, we construct the dataset from different domains of DeFi financial services, including *lending and borrowing, decentralized exchange, portfolio management, derivative*, and *aggregator*, which are the common types of DeFi applications. Moreover, we label the dataset by inspecting the source code and historical transactions of each DeFi protocol.

5.1.2. Dataset statistic and analysis

We have collected 99 open-source DeFi protocols into the dataset, which consists of a total of 7340 smart contracts. In the dataset, 58 DeFi protocols have experienced at least one external attack. Table 6 presents the statistics of the dataset. Specifically, 31 protocols are affected by a flash loan attack, of which *pump and arbitrage, price manipulation, and reentrancy* account for 26 %, 58 %, and 16 %, respectively. 4 protocols are struck by a deflation token attack. Additionally, 9 protocols have suffered from a DeFi sandwich attack, and 3 protocols have suffered a rug pull attack. There are also 11 protocols that have been influenced by other types of DeFi attacks.

Notably, this dataset reflects the actual landscape of DeFi exploits in the wild, where flash loan attacks dominate due to their low cost and flexibility. Nevertheless, we acknowledge that such an imbalance may bias evaluation metrics (e.g., overall accuracy), especially when modeling task performance across heterogeneous attack classes. In this context, we employ stratified sampling to maintain the proportional distribution of attack types across training and testing sets. We also report class-wise performance metrics to reflect the model robustness across different attack categories. In future work, we will expand the dataset with more balanced and diversified attack samples by incorporating threat intelligence feeds, on-chain monitoring data, and community-reported incidents. To embrace the DeFi community, we have released the dataset.

5.2. Selecting candidate tools

Not all of the studied tools are well suited for the experimental study. Only those tools that meet the following three criteria are included in the experiments: 1) the tool is fully open source and available; 2) the tool identifies vulnerabilities or risks in DeFi protocols or smart contracts; 3)

the tool is representative in its classification, e.g., frequent comparisons and usage as a baseline tool.

Note that this study focuses exclusively on evaluating vulnerability detection techniques and does not include the experiments with automated repair tools. This exclusion stems from both technical and practical constraints. Most deployed DeFi contracts on the mainnet are immutable, lack accessible upgrade mechanisms, and pose substantial risk when applying patches to live financial systems. Moreover, the absence of fully open-source patching frameworks and the lack of standardized evaluation methodologies further complicate a rigorous assessment of post-patch security and correctness. Future work will address these challenges by redeploying historically exploited contracts (e.g., The DAO, bZx, and Lendf.me) with patched variants in controlled testnet environments, enabling systematic and risk-free evaluation of automated repair techniques.

5.3. Evaluation metric

To measure the performance of the investigated tools, we adopt the widely used metrics, including *accuracy, precision, recall*, and *F1-score* Tian, Luo, Qiu, Du, and Guizani (2020). Each tool generates a bug report that contains the results of the vulnerability analysis. By matching it with the manually tagged labels of the DeFi protocols, we can obtain the true positive (TP), true negative (TN), false positive (FP), and false negative (FN) values of each tool. TP represents the number of data elements for which the tool correctly predicts samples with vulnerabilities. TN is the number of samples that the tool successfully detected without vulnerabilities. FP represents the number of samples that were falsely detected as having vulnerabilities. FN refers to the number of samples that were wrongly detected as protocols without vulnerabilities. Using these data, we further calculate the corresponding evaluation metrics as follows.

Accuracy. Accuracy is one of the most common metrics used to evaluate the pros and cons of the bug detection tools. Generally, it can objectively reflect the most direct ramification of a detection tool.

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN} \quad (1)$$

Precision. Precision is defined as the ratio of true positives and the total number of positives predicted by a tool. A greater precision value indicates a higher percentage of correct alarms and fewer false alarms, making bug verification and code modification easier.

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

Recall. Recall is the fraction of the total number of relevant instances that are actually detected. A higher recall value indicates that more real vulnerabilities are found, fewer hidden vulnerabilities are missed, and lower risk of unknown attacks.

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

F1-score. F1-score is the harmonic average of precision and recall, and it is usually treated as the critical evaluation indicator for some classification missions, e.g., bug detection.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (4)$$

Furthermore, we follow the settings of the previous works Durieux et al. (2020), Ren et al. (2021) and select the evaluation metric of execution time.

Execution Time. Execution time is a critical indicator of the efficiency of tools. Long scan times can compromise the practicality of these tools. To measure execution time, the latency, which represents the time interval between the start and end of the analysis, is recorded for each DeFi protocol. The evaluation shows the average execution time for each DeFi protocol in the dataset, which helps to assess the efficiency of the tools.

Table 7

Performance evaluation of DeFi security tools and traditional smart contract vulnerability detection tools in analyzing DeFi protocols in terms of accuracy, recall, precision, and F1-score.

#	Tools	Accuracy(%)	Recall(%)	Precision(%)	F1-score(%)
1	PROMUTATOR	68.45	72.30	68.69	70.45
2	DEFI-SCORE	73.27	75.94	63.14	68.95
3	SLITHER	58.37	4.50	83.33	8.54
4	SMARTCHECK	58.91	26.96	35.65	30.70
5	TEETHER	63.13	31.32	75.55	44.28
6	RECHECKER	50.48	86.02	43.28	57.59

Table 8

Scalability evaluation of traditional smart contract tools in analyzing DeFi protocols and smart contracts with different solc versions.

#	Tools	0.4.x	0.5.x	0.6.x	0.7.x	0.8.x	DeFi protocols
1	VERISOL	✓	×	×	×	×	×
2	MAIAN	✓	×	×	×	×	×
3	MANTICORE	✓	×	×	×	×	×
4	MYTHRIL	✓	×	×	×	×	×
5	OYENTE	✓	×	×	×	×	×
6	OSIRIS	✓	×	×	×	×	×
7	TEETHER	✓	✓	×	×	×	✓
8	VERX	✓	✓	×	×	×	×
9	RECHECKER	✓	✓	✓	✓	✓	✓
10	TMP	✓	×	×	×	×	×
11	SMARTCHECK	✓	✓	✓	✓	✓	✓
12	SECURIFY	✓	×	×	×	×	×
13	SLITHER	✓	✓	×	×	×	✓
14	CONTRACTFUZZER	✓	×	×	×	×	×
15	SFUZZ	✓	×	×	×	×	×
16	IR-FUZZ	✓	×	×	×	×	×
17	ILF	✓	✓	×	×	×	×
18	SMARTIAN	✓	×	×	×	×	×
19	CONFUZZIUS	✓	×	×	×	×	×

Given the class imbalance in our evaluation dataset, we report macro-averaged F1-scores to reduce the bias toward dominant classes. While confusion matrices and ROC curves could offer finer-grained insights into class-specific performance, our current evaluation focuses on summarizing overall effectiveness. We leave a detailed per-class analysis using these metrics as future work.

5.4. Results and analysis

In the following, we present the experimental results and analysis and provide answers to the research questions. We begin by verifying the performance of existing DeFi security tools in detecting vulnerabilities in DeFi protocols. Next, we evaluate the scalability of traditional smart contract bug detection tools in analyzing DeFi protocols and validate their effectiveness. In addition, we measure the overall efficiency of DeFi tools and traditional smart contract vulnerability detection tools running on DeFi protocols. Finally, we provide an in-depth analysis and suggest possible improvements for future research.

5.4.1. Performance evaluation (answer to RQ1)

Firstly, we conduct experiments to evaluate the performance of available DeFi tools in analyzing DeFi protocols. Experimental results of each tool are summarized in Table 7. From the table, we obtain the following observations. (1) Comparing different tools, DeFi-specific security tools gain a higher accuracy than other tools. For example, DEFI-SCORE and PROMUTATOR achieve accuracies of 73.27 % and 68.45 %, respectively, which is 10.14 % and 5.32 % higher than the traditional smart contract vulnerability detection tool TEETHER. (2) In terms of F1-Score, DEFI-SCORE and PROMUTATOR achieve 68.95 % and 70.45 % F1-score, outperforming other tools by a large margin. We attribute the superior performance of the DeFi-specific security tools to the ability to recover high-level semantics from DeFi protocols.

Table 9

Execution time evaluation of each studied tool in analyzing DeFi protocols.

#	Tools	Execution Time(s)	
		Average	Total
1	DEFI-SCORE	10.05	994.54
2	PROMUTATOR	11.11	1100.28
3	MAIAN	436.55	43218.67
4	MANTICORE	164.40	16275.90
5	OSIRIS	92.23	9130.77
6	MYTHRIL	126.62	12535.01
7	OYENTE	208.95	20686.05
8	TEETHER	331.89	32856.88
9	RECHECKER	8.02	793.95
10	SMARTCHECK	528.69	52339.97
11	SECURIFY	208.87	20678.13
12	SLITHER	216.26	21409.97
13	ILF	190.93	18902.03

5.4.2. Scalability evaluation (Answer to RQ2)

Next, we run all selected smart contract bug detection tools on the DeFi protocol dataset to determine whether they can effectively analyze complex DeFi protocols. To be more specific, we evaluate the scalability of each tool by testing it on both DeFi protocols and smart contracts written in different Solidity versions. Table 8 shows the experimental results. We can observe that most of the traditional vulnerability detection methods for smart contracts only support detecting low-version (i.e., solc-0.4.x) smart contracts, while only four of them are able to analyze DeFi protocols and generate detection results (i.e., TEETHER, SMARTCHECK, SLITHER, and RECHECKER).

Furthermore, we verify the effectiveness of the available tools in handling DeFi protocols, while Table 7 gives the evaluation results. It is evident that their analysis effect on the DeFi protocol is poor, and the F1-score of most tools is below 50 %. This can be attributed to the fact that traditional smart contract vulnerability detection tools have poor compatibility with complex intertwined contracts in DeFi protocols and cannot effectively capture defects in deep space.

5.4.3. Efficiency evaluation (Answer to RQ3)

In addition, we calculate the execution time required by the tools to analyze DeFi protocols. Our evaluation aims to provide an overall execution efficiency of DeFi security tools and smart contract analysis tools when running DeFi protocols. Note that the DeFi protocol dataset involves different contract types and application categories, and the DeFi protocols are randomly collected from Ethereum. For each tool, the experiment is fair, and the experimental results can reflect the execution efficiency of each tool on DeFi smart contracts.

Table 9 presents the total and average time used by each tool. The average execution time is per protocol. It considers the execution of the tool on a DeFi protocol, including compilation, construction of intermediate representation, analysis, and result parsing. From the table, we can observe two different groups of execution times: the tools that take a few seconds to execute and the tools that take a few minutes. For example, all the traditional vulnerability detection tools (lines 3–13 in Table 9) take a long time to analyze a DeFi protocol, ranging from 92.23s to 528.69s per protocol. In contrast, the DeFi-specific security tools such as DEFI-SCORE and PROMUTATOR are much faster, taking only 10.05s and 11.11s per protocol, respectively. This may stem from two facts: (1) traditional vulnerability detection tools still have inherent difficulties in understanding the complex logic of DeFi protocols; (2) the specially designed DeFi security tools are able to recover the high-level semantics of DeFi protocols.

Interestingly, the deep learning-based method RECHECKER is a fast tool that takes on average only 8.02s to analyze a DeFi protocol. We explain that the deep learning model spends a lot of time in the training phase, while the trained model can quickly identify the key characteristics of the DeFi protocols in the detection phase. However, we would

like to point out that the execution time does not reflect the complete picture of the performance of these tools. For example, most traditional vulnerability detection tools carry out invalid runs as their output is either empty or unavailable.

5.4.4. Summary and implication

Experimental results demonstrate that traditional smart contract vulnerability detection tools exhibit limited effectiveness when applied to DeFi protocols. These tools often fail to capture the complex inter-contract interactions and domain-specific logic inherent to DeFi systems, leading to poor detection performance. In contrast, DeFi-specific security tools show greater promise, particularly in identifying known attack behaviors. However, their capabilities remain restricted in several dimensions: they struggle to generalize across diverse protocol designs, exhibit limited coverage of high-level logic vulnerabilities, and often rely on hardcoded heuristics or known exploit patterns. These limitations highlight a fundamental gap between existing detection techniques and the nuanced security requirements of DeFi ecosystems.

Our findings validate the need for detection frameworks that incorporate deeper semantic understanding, support for multi-contract reasoning, and adaptability to novel attack vectors. At the same time, current tools lack formal guarantees and often yield incomplete or inconsistent analysis results, especially when evaluating complex DeFi transactions involving asynchronous state changes or composable protocol interactions. These observations underscore the need of developing advanced, semantics-aware, and provably sound vulnerability detection tools tailored to the unique operational models of DeFi protocols.

6. Discussion, conclusion and outlook

6.1. Discussion

While interest in DeFi security is growing, and considerable progress has been made in this area over the past few years (Chaliasos et al., 2023), there are still outstanding issues that require attention in future research endeavors. In this subsection, we discuss the primary challenges faced by users and developers within the interconnected DeFi landscape.

6.1.1. Composability

Composability stands as a fundamental pillar of decentralized finance, enabling applications and protocols to interact with each other in a permissionless manner (Saengchote, 2021). Leveraging the open nature of DeFi, these applications synergistically combine to create new forms of financial services previously unimaginable. For example, in DeFi, cryptoassets can be effortlessly tokenized and exchanged between different DeFi protocols, akin to the concept of rehypothecation in traditional finance.

However, the very feature that empowers DeFi's composability also exposes traders and protocols to substantial risks (Gudgeon, Perez, Harz, Livshits, & Gervais, 2020). In particular, composability can allow attackers to orchestrate complex multi-step transactions across multiple protocols within a single block, making it difficult to trace or intercept. A representative case is the bZx flash loan exploit, in which the attacker borrowed a large amount of ETH via flash loan from dYdX, used it to manipulate the pricing oracle of bZx by trading on Uniswap, and then took out an undercollateralized loan from bZx. This attack spanned at least three protocols - dYdX, Uniswap, and bZx - and succeeded due to flawed assumptions about atomicity and price integrity across platforms. Another well-known example is the Harvest Finance attack, where an attacker manipulated the value of Curve LP tokens by rapidly swapping large volumes through Curve Finance, which in turn misled Harvest's pricing oracle. The attacker then deposited overvalued assets into Harvest's vault and immediately withdrew a larger amount than they had deposited. Again, this exploit relied on inter-protocol composition and inconsistent state assumptions.

To address composability risks, recent efforts (Tolmach, Li, Lin, & Liu, 2021) have proposed a process-algebraic technique to achieve property verification by modeling DeFi protocols in a compositional manner. Nevertheless, a significant gap persists in DeFi research, particularly in the formalization of security models that can detect and reason about emergent behavior arising from protocol composition. Despite being a foundational aspect of DeFi, composability continues to present major security challenges for protocol designers as the ecosystem grows in complexity and scale.

6.1.2. Underlying infrastructure

Another critical security issue in DeFi is associated with the underlying blockchain. Since DeFi applications run on top of the blockchain system, their execution is affected by potential blockchain vulnerabilities. Due to the open and transparent nature of the blockchain system, all market states of DeFi protocols can be publicly observed in real-time. Meanwhile, users' transactions are broadcast to the blockchain network and stored in a mempool. The transactions in this mempool can be accessed by anyone before miners package them in a block. Therefore, attackers can forecast the future market states based on non-executed transactions and utilize this knowledge to manipulate the market operation. For instance, attackers may observe a victim transaction in the mempool and then front-run or back-run them to gain profit (Zhou, Qin, & Gervais, 2021b). Moreover, DeFi protocols are isolated from the outside of the blockchain system, while external information cannot be utilized by these applications in real time. As a result, DeFi protocols rely on each other and share information to maintain the market operation. For example, some lending borrowing protocols utilize the real-time exchange rate in DEXs to determine the collateral rate. Nevertheless, some attackers may target dependencies between platforms by manipulating the state of one application and profiting from another one (Zhou, Qin, Cully, Livshits, & Gervais, 2021a). Security challenges related to DeFi protocols involve interactions between traders and DeFi protocols that are difficult to effectively detect before applications are launched. Such security issues are associated with DeFi markets and have a long-term and continuous impact on DeFi users.

6.2. Conclusion and outlook

As the DeFi ecosystem continues to expand and find applications in an ever-growing number of domains, the associated security concerns, including DeFi attacks, are expected to escalate in the coming years. DeFi has already seen a substantial number of attacks, resulting in billions of dollars in financial losses, thereby posing a significant threat to the overall DeFi ecosystem security.

This paper aims to provide a comprehensive understanding of various DeFi vulnerabilities and attacks, and present an empirical review of state-of-the-art techniques capable of detecting vulnerabilities, identifying attacks, assessing risk, and fixing defects in DeFi protocols and smart contracts. A review of the existing literature reveals a variety of techniques that have been proposed over the years to automatically identify vulnerabilities in smart contracts. However, a significant limitation is that most of them are designed to handle traditional smart contracts or single contracts, making them adept at detecting low-level bugs in smart contracts but ill-suited for analyzing complex DeFi protocols.

While recent endeavors have shown promise in capturing attack patterns in DeFi protocols by extracting high-level DeFi semantics from raw transactions, they still struggle to effectively differentiate compound vulnerabilities and attacks. Furthermore, the immutability inherent in the underlying blockchain infrastructure poses challenges when attempting to patch vulnerabilities in on-chain contracts. Current automated repair tools can address only certain types of vulnerabilities and face difficulties in handling complex DeFi protocols.

There remain several key issues that need to be addressed in the future to broaden the applicability of these tools in various domains such

as vulnerability detection, attack hunting, risk assessment, and automated repair. As mentioned earlier, comprehending the interdependencies within the complicated web of DeFi space is critical for deciphering the logical rules underpinning DeFi protocols. Additionally, improving scalability is imperative to enable existing analysis tools to support a wider range of attack detection and automated repair scenarios. These challenges and related issues are expected to be at the forefront of research priorities in the field of DeFi ecosystem security.

CRedit authorship contribution statement

- Peng Qian: Writing and polishing the paper, Experiment design.
- Rui Cao: Data curation, Collect literature, Visualization.
- Zhenguang Liu: Polishing the paper.
- Wenqing Li: Collect and classify literature.
- Ming Li: Polishing the paper.
- Lun Zhang: Conducting experiment.
- Yufeng Xu: Visualization, Polishing the paper.
- Jianhai Chen: Supervision, Validation.
- Qinming He: Supervision.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the "Pioneer" and "Leading Goose" R&D Program of Zhejiang under Grant GS0012025C01084.

References

- Abdulhakeem, S. A., Hu, Q. et al. (2021). Powered by blockchain technology, DeFi (decentralized finance) strives to increase financial inclusion of the unbanked by reshaping the world financial system. *Modern Economy*, 12(01), 1.
- Agbo, C. C., Mahmoud, Q. H., & Eklund, J. M. (2019). Blockchain technology in healthcare: A systematic review. In *Healthcare* (p. 56). Multidisciplinary Digital Publishing Institute (vol. 7).
- Akropolis (2023). Delphi savings pool exploit. Accessed: 08-08-2023 Website. <https://akropolis.substack.com/p/delphi-savings-pool-exploit?s=r>.
- Albert, E., Gordillo, P., Livshits, B., Rubio, A., & Sergey, I. (2018). EthIR: A framework for high-level analysis of Ethereum bytecode. In *International symposium on automated technology for verification and analysis* (pp. 513–520). Springer.
- Amler, H., Eckey, L., Faust, S., Kaiser, M., Sandner, P., & Schlosser, B. (2021). DeFi-ing DeFi: Challenges & pathway. In *2021 3rd conference on blockchain research & applications for innovative networks and services (BRAINS)* (pp. 181–184). IEEE.
- Anyswap, M. P. (2023). Anyswap multichain router v3 exploit statement. Accessed: 08-08-2023 Website. <https://medium.com/multichainorg/anyswap-multichain-router-v3-exploit-statement-6833f1b7e6fb>.
- Atzei, N., Bartoletti, M., & Cimoli, T. (2017). A survey of attacks on Ethereum smart contracts (SoK). In *Principles of security and trust: 6th international conference, POST 2017, held as part of the european joint conferences on theory and practice of software, ETAPS 2017, Uppsala, Sweden, April 22–29, 2017, proceedings 6* (pp. 164–186). Springer.
- autofarm.network (2023). Venus vaults post-mortem. Accessed: 08-08-2023 Website. <https://medium.com/autofarm-network/21-april-2021-venus-vaults-post-mortem-1518ae7399c6>.
- Bamakan, S. M. H., Motavali, A., & Bondarti, A. B. (2020). A survey of blockchain consensus algorithms performance evaluation criteria. *Expert Systems with Applications*, 154, 113385.
- Bartoletti, M., Chiang, H.-y., James, & Lluch-Lafuente, A. (2021a). Maximizing extractable value from automated market makers. arXiv preprint arXiv:2106.01870.
- Bartoletti, M., Chiang, J. H.-y., & Lafuente, A. L. (2021b). SoK: Lending pools in decentralized finance. In *International conference on financial cryptography and data security* (pp. 553–578). Springer.
- Bartoletti, M., Chiang, J. H.-y., & Lafuente, A. L. (2021c). Towards a theory of decentralized finance. In *International conference on financial cryptography and data security* (pp. 227–232). Springer.
- Bartoletti, M., Chiang, J. H.-y., & Lluch-Lafuente, A. (2021d). A theory of automated market makers in DeFi. In *International conference on coordination languages and models* (pp. 168–187). Springer.
- Benson, J. (2023a). Ethereum DeFi protocol beanstalk hacked for \$182 million—what you need to know. Website. <https://decrypt.co/98118/ethereum-defi-protocol-beanstalk-hacked-182-million-what-you-need-know>.
- Benson, J. (2023b). Grim finance hacked for \$30 million in fantom tokens. Accessed: 08-08-2023 Website. <https://decrypt.co/88727/grim-finance-hacked-30-million-fantom-tokens>.
- BlockSec (2023a). The analysis of nerve bridge security incident. Website. Accessed: 08-08-2023. <https://blocksecteam.medium.com/the-analysis-of-nerve-bridge-security-incident-ead361a21025>.
- BlockSec (2023b). The analysis of the sanshu inu security incident. Accessed: 08-08-2023 Website. <https://blocksecteam.medium.com/the-analysis-of-the-sanshu-inu-security-incident-28c0c7c0e783>.
- BlockSec (2023c). Deposit less, get more: yCREDIT attack details. Website. Accessed: 08-08-2023. <https://blocksecteam.medium.com/the-analysis-of-the-sanshu-inu-security-incident-28c0c7c0e783>.
- Boyle, P. (2023). Saddle finance loses over \$10 million in hack. Website. Accessed: 08-08-2023. <https://blocksecteam.medium.com/the-analysis-of-the-sanshu-inu-security-incident-28c0c7c0e783>.
- Brent, L., Jurisevic, A., Kong, M., Liu, E., Gauthier, F., Gramoli, V., Holz, R., & Scholz, B. (2018). Vandal: A scalable security analysis framework for smart contracts. arXiv preprint arXiv:1809.03981.
- Buterin, V. et al. (2019). A next-generation smart contract and decentralized application platform. 2014. <https://github.com/ethereum/wiki/wiki/White-Paper>
- Cao, Y., Zou, C., & Cheng, X. (2021). Flashot: a snapshot of flash loan attack on DeFi ecosystem. arXiv preprint arXiv:2102.00626.
- CertiK (2023). Fei protocol incident analysis. Accessed: 08-08-2023 Website. <https://certik.medium.com/fei-protocol-incident-analysis-8527440696cc>.
- ChainSwap (2023a). Chainswap exploit 11 July 2021 post-mortem. Accessed: 08-08-2023 Website. <https://chain-swap.medium.com/chainswap-exploit-11-july-2021-post-mortem-6e4e346e5a32>.
- ChainSwap (2023b). Chainswap post-mortem and compensation plan. Accessed: 08-08-2023 Website. <https://chain-swap.medium.com/chainswap-post-mortem-and-compensation-plan-90cad50898ab>.
- Chakraborty, S., Krishna, R., Ding, Y., & Ray, B. (2022). Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering*, 48(09), 3280–3296.
- Chaliasos, S., Charalambous, M. A., Zhou, L., Galanopoulou, R., Gervais, A., Mitropoulos, D., & Livshits, B. (2023). Smart contract and DeFi security: Insights from tool evaluations and practitioner surveys.
- Chaliasos, S., Charalambous, M. A., Zhou, L., Galanopoulou, R., Gervais, A., Mitropoulos, D., & Livshits, B. (2024). Smart contract and DeFi security tools: Do they meet the needs of practitioners? In *Proceedings of the 46th IEEE/ACM international conference on software engineering* (pp. 1–13).
- Chen, J., Xia, X., Lo, D., Grundy, J., Luo, X., & Chen, T. (2022). DefectChecker: Automated smart contract defect detection by analyzing EVM bytecode. *IEEE Transactions on Software Engineering*, 48(07), 2189–2207.
- Chen, W., Xu, Z., Shi, S., Zhao, Y., & Zhao, J. (2018). A survey of blockchain applications in different domains. In *Proceedings of the 2018 international conference on blockchain technology and application* (pp. 17–21).
- Chen, Y.-P., & Ko, J.-C. (2019). CryptoAR wallet: A blockchain cryptocurrency wallet application that uses augmented reality for on-chain user data display. In *Proceedings of the 21st international conference on human-computer interaction with mobile devices and services* (pp. 1–5).
- Chohan, U. W. (2021). Decentralized finance (DeFi): An emergent alternative financial architecture. *Critical Blockchain Research Initiative (CBRI) Working Papers*.
- Choi, J., Kim, D., Kim, S., Grieco, G., Groce, A., & Cha, S. K. (2021). SMARTIAN: Enhancing smart contract fuzzing with static and dynamic data-flow analyses. In *2021 36th IEEE/ACM international conference on automated software engineering (ASE)* (pp. 227–239). IEEE.
- Chu, H., Zhang, P., Dong, H., Xiao, Y., Ji, S., & Li, W. (2023). A survey on smart contract vulnerabilities: Data sources, detection and repair. *Information and Software Technology*, 159 (p. 107221).
- Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L., & Juels, A. (2020). Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *2020 IEEE symposium on security and privacy (SP)* (pp. 910–927). IEEE.
- Dalton, M. (2023). Chainlink endures spam attack: Congestion, high fees. Accessed: 08-08-2023 Website. <https://cryptobriefing.com/chainlink-endures-spam-attack-congestion-high-fees/>.
- DAO, M. (2023). Bondly exploit—how it unfolded on zenterest postmortem. Accessed: 08-08-2023 Website. <https://medium.com/mantra-dao/bondly-exploit-how-it-unfolded-on-zenterest-postmortem-d8120d8d784b>.
- Deng, W., Wei, H., Huang, T., Cao, C., Peng, Y., & Hu, X. (2023). Smart contract vulnerability detection based on deep learning and multimodal decision fusion. *Sensors*, 23(16).
- DODO (2023). Dodo pool incident postmortem: With a little help from our friends. Accessed: 08-08-2023 Website. <https://blog.dodoex.io/dodo-pool-incident-postmortem-with-a-little-help-from-our-friends-327e66872d42>.
- Dollar, S. (2023). Saddle finance loses over \$10 million in hack. Accessed: 08-08-2023 Website. <https://safedollar.medium.com/safedollar-post-mortem-analysis-cb2769fe059>.
- DPO (2014). Delegated proof-of-stake (DPoS). White Paper.

- Durieux, T., Ferreira, J. F., Abreu, R., & Cruz, P. (2020). Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In *Proceedings of the ACM/IEEE 42nd international conference on software engineering* (pp. 530–541).
- EasyFi (2023). Easyfi security incident. pre-post mortem. Accessed: 08-08-2023 Website. <http://web.archive.org/web/20210420195832/https://medium.com/easyfi-network/easyfi-security-incident-pre-post-mortem-33f2942016e9>.
- Feist, J., Grieco, G., & Groce, A. (2019). Slither: A static analysis framework for smart contracts. In *2019 IEEE/ACM 2nd international workshop on emerging trends in software engineering for blockchain (WETSEB)* (pp. 8–15). IEEE.
- Ferreira Torres, C., Jonker, H., & State, R. (2022). Elysium: Context-aware bytecode-level patching to automatically heal vulnerable smart contracts. In *Proceedings of the 25th international symposium on research in attacks, intrusions and defenses* (pp. 115–128).
- Finance, B. (2023a). May 29, incident report. Accessed: 08-08-2023 Website. <https://medium.com/belt-finance/may-29-incident-report-865d20cc46ca>.
- Finance, B. (2023b). PolyBUNNY post-mortem & compensation. Accessed: 08-08-2023 Website. <https://pancakebunny.medium.com/polybunny-post-mortem-compensation-42b5c35ce957>.
- Finance, B. (2023c). Post mortem analysis. Accessed: 08-08-2023 Website. <https://pancakebunny.medium.com/hello-bunny-fam-a7bf0c7a07ba>.
- Finance, H. (2023d). Harvest flash loan economic attack post-mortem. Accessed: 08-08-2023 Website. <https://medium.com/harvest-finance/harvest-flashloan-economic-attack-post-mortem-3cf900d65217>.
- Finance, I. (2023e). Indexed attack post-mortem. Accessed: 08-08-2023 Website. <https://ndxfi.medium.com/indexed-attack-post-mortem-b006094f0bdc>.
- Finance, V. (2023f). Vee.finance incident announcement. Accessed: 08-08-2023 Website. <https://veefi.medium.com/vee-finance-accident-announcement-5e75f197da6>.
- Finance, W. (2023g). Warp finance—exploit summary & recovery of funds. Accessed: 08-08-2023 Website. <https://warpfinance.medium.com/warp-finance-exploit-summary-recovery-of-funds-5b8fe4a11898>.
- Foundation, S. (2023). Uranium finance exploit analysis. Accessed: 08-08-2023 Website. <https://medium.com/shentu-foundation/uranium-finance-exploit-analysis-d135055d6a6a>.
- FURUCOMBO (2023). Furucombo post-mortem march 2021. Accessed: 08-08-2023 Website. <https://medium.com/furucombo/furucombo-post-mortem-march-2021-ad19afd415e>.
- Gagliardi, T. (2023). The poly network hack explained. Accessed: 08-08-2023 Website. <https://research.kudelskisecurity.com/2021/08/12/the-poly-network-hack-explained/>.
- Genç, E. (2023). Burgerswap explains \$7.2 million flash loan attack in post-mortem. Accessed: 08-08-2023 Website. <https://decrypt.co/72194/burgerswap-explains-7-2-million-flash-loan-attack-in-post-mortem>.
- Gervais, A., Karame, G. O., Wüst, K., Glykantzis, V., Ritzdorf, H., & Capkun, S. (2016). On the security and performance of proof of work blockchains. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security* (pp. 3–16).
- Giesen, J.-R., Andreina, S., Rodler, M., Karame, G. O., & Davi, L. (2022). Practical mitigation of smart contract bugs. arXiv preprint arXiv:2203.00364.
- Gogo, J. (2023). \$160m at risk due to bug in DeFi lending protocol compound. Accessed: 08-08-2023 Website. <https://decrypt.co/82499/compound-exploit-drains-21m-from-lending-protocol>.
- Gottsegen, W. (2023). DeFi protocol pickle finance hacked for \$20 million. Accessed: 08-08-2023 Website. <https://decrypt.co/49149/pickle-finance-hack>.
- Group, A. (2023). Preventing re-entrancy attacks—lessons from history. Accessed: 08-08-2023 Website. <https://medium.com/amber-group/preventing-re-entrancy-attacks-lessons-from-history-c2d96480fac3>.
- Gu, R., Marinescu, R., Seceleanu, C., & Lundqvist, K. (2018). Formal verification of an autonomous wheel loader by model checking. In *Proceedings of the 6th conference on formal methods in software engineering* (pp. 74–83).
- Gudgeon, L., Perez, D., Harz, D., Livshits, B., & Gervais, A. (2020). The decentralized financial crisis. In *2020 crypto valley conference on blockchain technology (CVCBT)* (pp. 1–15). IEEE.
- Halborn (2023). Explained: The badgerdao hack. Accessed: 08-08-2023 Website. <https://halborn.com/explained-the-badgerdao-hack-december-2021/>.
- Hayward, A. (2023). Hacker drains \$622m from axie infinity's ronin Ethereum sidechain. Accessed: 08-08-2023 Website. <https://decrypt.co/96322/hacker-622-million-axie-infinity-ronin-ethereum>.
- He, D., Deng, Z., Zhang, Y., Chan, S., Cheng, Y., & Guizani, N. (2020). Smart contract vulnerability analysis and security audit. *IEEE Network*, 34(5), 276–282.
- He, J., Balunović, M., Ambroladze, N., Tsankov, P., & Vechev, M. (2019). Learning to fuzz from symbolic execution with application to smart contracts. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security* (pp. 531–548).
- Hu, T., Liu, X., Chen, T., Zhang, X., Huang, X., Niu, W., Lu, J., Zhou, K., & Liu, Y. (2021). Transaction-based classification and detection approach for Ethereum smart contract. *Information Processing & Management*, 58(2), 102462.
- Hwang, S.-J., Choi, S.-H., Shin, J., & Choi, Y.-H. (2022). CodeNet: Code-targeted convolutional neural network architecture for smart contract vulnerability detection. *IEEE Access*, 10, 32595–32607.
- Initiative, Q. (2023). Cryptocurrency exchange hacks, frauds, and scams. Accessed: 08-08-2023 Website. <https://www.quadriginitiative.com/education.php>.
- Inspx (2023). Reentrancy attack on cream finance. Accessed: 08-08-2023 Website. <https://inspxco.medium.com/reentrancy-attack-on-cream-finance-incident-analysis-1c629686b6f5>.
- Jensen, J. R., von Wachter, V., & Ross, O. (2021). An introduction to decentralized finance (DeFi). *Complex Systems Informatics and Modeling Quarterly*, (26), 46–54.
- Jiang, B., Liu, Y., & Chan, W. K. (2018). ContractFuzzer: Fuzzing smart contracts for vulnerability detection. In *Proceedings of the 33rd ACM/IEEE International conference on automated software engineering* (pp. 259–269). ACM.
- Jin, H., Wang, Z., Wen, M., Dai, W., Zhu, Y., & Zou, D. (2021). Aroc: An automatic repair framework for on-chain smart contracts. *IEEE Transactions on Software Engineering*, 48(11), 4611–4629.
- Jordan, L., & Jack, C. (2023). DeFi score: An open framework for evaluating DeFi protocols. Accessed: 08-08-2023 Website. <https://github.com/ConsenSys/defi-score>.
- JustLiquidity (2023). Flash loan farming / JULB / BNB. Accessed: 08-08-2023 Website. <https://justliquidity.medium.com/flash-loan-farming-julb-bnb-14c6c128f5dd>.
- Kalra, S., Goel, S., Dhawan, M., & Sharma, S. (2018). Zeus: Analyzing safety of smart contracts. In *Ndss* (pp. 1–12).
- King, S., & Nadal, S. (2012). Ppcoin: Peer-to-peer crypto-currency with proof-of-stake. *self-published paper*, August, 19(1).
- Kistner, K. J. (2023a). token duplication incident report. Accessed: 08-08-2023 Website. <https://bzx.network/blog/incident>.
- Kistner, K. J. (2023b). Post-mortem. Accessed: 08-08-2023 Website. <https://bzx-network/blog/postmortem-ethdenver>.
- Kong, Q., Chen, J., Wang, Y., Jiang, Z., & Zheng, Z. (2023). Defitainter: Detecting price manipulation vulnerabilities in DeFi protocols. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis* (pp. 1144–1156).
- Krupp, J., & Rossow, C. (2018). {teEther}: Gnawing at Ethereum to automatically exploit smart contracts. In *27th USENIX security symposium (USENIX security 18)* (pp. 1317–1333).
- Li, W., Bu, J., Li, X., & Chen, X. (2022a). Security analysis of DeFi: Vulnerabilities, attacks and advances. In *2022 IEEE international conference on blockchain (blockchain)* (pp. 488–493). IEEE.
- Li, W., Bu, J., Li, X., Peng, H., Niu, Y., & Zhang, Y. (2022b). A survey of DeFi security: Challenges and opportunities. *Journal of King Saud University-Computer and Information Sciences*, 34(10), 10378–10404.
- Lin, G., Wen, S., Han, Q.-L., Zhang, J., & Xiang, Y. (2020). Software vulnerability detection using deep neural networks: A survey. *Proceedings of the IEEE*, 108(10), 1825–1848.
- Liu, C., Liu, H., Cao, Z., Chen, Z., Chen, B., & Roscoe, B. (2018a). Reguard: Finding reentrancy bugs in smart contracts. In *2018 IEEE/ACM 40th international conference on software engineering: Companion (ICSE-companion)* (pp. 65–68). IEEE.
- Liu, H., Liu, C., Zhao, W., Jiang, Y., & Sun, J. (2018b). S-Gram: Towards semantic-aware security auditing for Ethereum smart contracts. In *2018 33rd IEEE/ACM international conference on automated software engineering (ASE)* (pp. 814–819). IEEE.
- LIU (2021). Fixed degree of decentralization DPos consensus mechanism in blockchain based on adjacency vote and the average fuzziness of vague value. *Computer Networks*, 199, 108432. <https://doi.org/10.1016/j.comnet.2021.108432>
- Liu, M. (2023). Urgent: OUSD was hacked and there has been a loss of funds. Accessed: 08-08-2023 Website. <https://blog.originprotocol.com/urgent-ousd-has-hacked-and-there-has-been-a-loss-of-funds-7b8c4a7d534c>.
- Liu, Y., Lu, Q., Zhu, L., Paik, H.-Y., & Staples, M. (2023a). A systematic literature review on blockchain governance. *Journal of Systems and Software*, 197, 111576.
- Liu, Z., Qian, P., Wang, X., Zhu, L., He, Q., & Ji, S. (2021). Smart contract vulnerability detection: From pure neural network to interpretable graph feature and expert pattern fusion. In *Ijcai* (pp. 2751–2759).
- Liu, Z., Qian, P., Wang, X., Zhuang, Y., Qiu, L., & Wang, X. (2023b). Combining graph neural networks with expert knowledge for smart contract vulnerability detection. *IEEE Transactions on Knowledge and Data Engineering*, 35(2), 1296–1310.
- Liu, Z., Qian, P., Yang, J., Liu, L., Xu, X., He, Q., & Zhang, X. (2023c). Rethinking smart contract fuzzing: Fuzzing with invocation ordering and important branch revisiting. *IEEE Transactions on Information Forensics and Security*, 18, 1237–1251.
- Lucid, D. (2023). Rari capital Ethereum pool—post-mortem. Accessed: 08-08-2023 Website. <https://medium.com/rari-capital/5-8-2021-rari-ethereum-pool-post-mortem-60aab6a6f8f9>.
- Luu, L., Chu, D.-H., Olickel, H., Saxena, P., & Hobor, A. (2016). Making smart contracts smarter. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security* (pp. 254–269). ACM.
- Maiboroda, M. (2023). Another huge DeFi-exploit, ankr protocol attacked for 10 trillion aBNB tokens. Accessed: 08-08-2023 Website. <https://boxmining.com/attack-on-ankr-abnbc/>.
- MakerDAO (2023). The market collapse of march 12–13, 2020: How it impacted makerDAO. Accessed: 08-08-2023 Website. <https://blog.makerdao.com/the-market-collapse-of-march-12-2020-how-it-impacted-makerdao/>.
- Malwa, S. (2023a). How market manipulation led to \$100m exploit on solana DeFi exchange mango. Accessed: 08-08-2023 Website. <https://www.coindesk.com/markets/2022/10/12/how-market-manipulation-led-to-a-100m-exploit-on-solana-defi-exchange-mango/>.
- Malwa, S. (2023b). Rug pull scams. Accessed: 08-08-2023 Website. <https://www.coindesk.com/markets/2021/12/17/defi-rug-pull-scams-pulled-in-28b-this-year-chainalysis>.
- McDonald, M. (2023). Incident with non-standard ERC20 deflationary tokens. Accessed: 08-08-2023 Website. <https://medium.com/balancer-protocol/incident-with-non-standard-erc20-deflationary-tokens-95a0f6d46dea>.
- Moosavi, J., Naeni, L. M., Fathollahi-Fard, A. M., & Fiore, U. (2021). Blockchain in supply chain management: A review, bibliometric, and network analysis. *Environmental Science and Pollution Research*, 1, (pp. 1–15).
- Mossberg, M., Manzano, F., Hennenfent, E., Groce, A., Grieco, G., Feist, J., Brunson, T., & Dinaburg, A. (2019). Manticore: A user-friendly symbolic execution framework for binaries and smart contracts. In *2019 34th IEEE/ACM international conference on automated software engineering (ASE)* (pp. 1186–1189). IEEE.
- Mueller, B. (2023). A framework for bug hunting on the Ethereum blockchain. Accessed: 08-08-2023 Website. <https://github.com/ConsenSys/mythril>.
- Murray, Y., & Anisi, D. A. (2019). Survey of formal verification methods for smart contracts on blockchain. In *2019 10th IFIP international conference on new technologies, mobility and security (NTMS)* (pp. 1–6). IEEE.

- Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system. *Decentralized Business Review*, 1, (p. 21260).
- NETWORK, P. (2023). Paid network attack postmortem. Accessed: 08-08-2023 Website. <https://paidnetwork.medium.com/paid-network-attack-postmortem-march-7-2021-9e4c0fe0e07>.
- Nguyen, T. D., Pham, L. H., & Sun, J. (2021a). SGUARD: Towards fixing vulnerable smart contracts automatically. In *42nd IEEE symposium on security and privacy, SP 2021, San Francisco, CA, USA, 24–27 may 2021* (pp. 1215–1229). IEEE. <https://doi.org/10.1109/SP40001.2021.00057>
- Nguyen, T. D., Pham, L. H., & Sun, J. (2021b). SGUARD: Towards fixing vulnerable smart contracts automatically. In *2021 IEEE Symposium on security and privacy (SP)* (pp. 1215–1229). IEEE.
- Nguyen, T. D., Pham, L. H., Sun, J., Lin, Y., & Minh, Q. T. (2020). Sfuzz: An efficient adaptive fuzzer for solidity smart contracts. In *Proceedings of the ACM/IEEE 42nd international conference on software engineering* (pp. 778–788).
- Nikolić, I., Kolluri, A., Sergey, I., Saxena, P., & Hobor, A. (2018). Finding the greedy, prodigal, and suicidal contracts at scale. In *Proceedings of the 34th annual computer security applications conference* (pp. 653–663).
- Oosthoek, K. (2021). Flash crash for cash: Cyber threats in decentralized finance. arXiv preprint arXiv:2106.10740.
- PeckShield (2023a). Opyin hacks: Root cause analysis. Accessed: 08-08-2023 Website. <https://peckshield.medium.com/opyin-hacks-root-cause-analysis-c65f3fe249db>.
- PeckShield (2023b). Uniswap/lendf.me hacks: Root cause and loss analysis. Accessed: 08-08-2023 Website. <https://peckshield.medium.com/uniswap-lendf-me-hacks-root-cause-and-loss-analysis-50f3263dccc9>.
- Perez, D., & Livshits, B. (2019). Smart contract vulnerabilities: Does anyone care? (pp. 1–15). arXiv preprint arXiv:1902.06710.
- Perez, D., & Livshits, B. (2021). Smart contract vulnerabilities: Vulnerable does not imply exploited. In *30th USENIX security symposium (USENIX security 21)* (pp. 1325–1341).
- Permev, A., Dimitrov, D., Tsankov, P., Drachler-Cohen, D., & Vechev, M. (2020). VerX: Safety verification of smart contracts. In *2020 IEEE symposium on security and privacy (SP)* (pp. 1661–1677). IEEE.
- Popsicle.Finance (2023). Popsicle finance post mortem- after fragola hack. Accessed: 08-08-2023 Website. <https://popsiclefinance.medium.com/popsicle-finance-post-mortem-after-fragola-hack-f45b302362e0>.
- Protocol, C. (2023). 12/28 post-mortem. Accessed: 08-08-2023 Website. <https://cover-protocol.medium.com/12-28-post-mortem-34c5f9f718d4>.
- Qian, P., He, J., Liu, L., Wu, S., Liu, Z., Wu, L., Zhou, Y., & He, Q. (2023a). Demystifying random number in Ethereum smart contract: Taxonomy, vulnerability identification, and attack detection. *IEEE Transactions on Software Engineering*, 49(7), 3793–3810.
- Qian, P., Liu, Z., He, Q., Huang, B., Tian, D., & Wang, X. (2022). Smart contract vulnerability detection technique: A survey. arXiv preprint arXiv:2209.05872.
- Qian, P., Liu, Z., He, Q., Zimmermann, R., & Wang, X. (2020). Towards automated reentrancy detection for smart contracts based on sequential models. *IEEE Access*, 8, 19685–19695.
- Qian, P., Liu, Z., Yin, Y., & He, Q. (2023b). Cross-modality mutual learning for enhancing smart contract vulnerability detection on bytecode. In *Proceedings of the ACM web conference 2023* (pp. 2220–2229).
- Qian, P., Wu, H., Du, Z., Vural, T., Rong, D., Cao, Z., Zhang, L., Wang, Y., Chen, J., & He, Q. (2023c). Mufuzz: Sequence-aware mutation and seed mask guidance for blockchain smart contract fuzzing. arXiv preprint arXiv:2312.04512.
- Qin, K., Zhou, L., Afonin, Y., Lazzaretti, L., & Gervais, A. (2021a). Cefi vs. DeFi-comparing centralized to decentralized finance. arXiv preprint arXiv:2106.08157.
- Qin, K., Zhou, L., & Gervais, A. (2021b). Quantifying blockchain extractable value: How dark is the forest? arXiv preprint arXiv:2101.05511.
- Qin, K., Zhou, L., Livshits, B., & Gervais, A. (2021c). Attacking the DeFi ecosystem with flash loans for fun and profit. In *International conference on financial cryptography and data security* (pp. 3–32). Springer.
- Rameder, H., Di Angelo, M., & Salzer, G. (2022). Review of automated vulnerability analysis of smart contracts on Ethereum. *Frontiers in Blockchain*, 5, 814977.
- REKT (2023). Eminence - rekt in prod. Accessed: 08-08-2023 Website. <https://rekt.news/eminence-rekt-in-prod/>.
- REKT (2023). Rekt news. Accessed: 08-08-2023 Website. <https://de.fi/rekt-database>.
- REKT (2023a). Value DeFi (1). Accessed: 08-08-2023 Website. <https://rekt.news/value-defi-rekt/>.
- REKT (2023b). Value DeFi (2). Accessed: 08-08-2023 Website. <https://rekt.news/value-rekt2/>.
- REKT (2023c). Value DeFi (3). Accessed: 08-08-2023 Website. <https://rekt.news/value-rekt3/>.
- Ren, M., Yin, Z., Ma, F., Xu, Z., Jiang, Y., Sun, C., Li, H., & Cai, Y. (2021). Empirical evaluation of smart contract testing: What is the best choice? In *Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis ISSTA 2021* (p. 566–579).
- Rex, H., Lucas, H., Mohamed, A., David, D., Nick, S., Nic, V., & Carter, C. (2023). DeFi-safety is an independent ratings organization that evaluates decentralized finance products. Accessed: 08-08-2023 Website. <https://www.defisafety.com/>.
- Rodler, M., Li, W., Karame, G. O., & Davi, L. (2018). Sereum: Protecting existing smart contracts against re-entrancy attacks. arXiv preprint arXiv:1812.05934.
- Rodler, M., Li, W., Karame, G. O., & Davi, L. (2021). {EVMPatch}: Timely and automated patching of Ethereum smart contracts. In *30th USENIX security symposium (USENIX security 21)* (pp. 1289–1306).
- Russell, J. (2023). The crypto world's latest hack sees bancor lose \$23.5m. Accessed: 08-08-2023 Website. <https://techcrunch.com/2018/07/10/bancor-loses-23-5m/>.
- Saengchote, K. (2021). Where do DeFi stablecoins go? a closer look at what DeFi composability really means. *A closer look at what DeFi composability really means*. (July 26, 2021).
- Sam Kessler, B. B. (2023). Crypto bridge nomad drained of nearly \$200m in exploit. Accessed: 08-08-2023 Website. <https://www.coindesk.com/tech/2022/08/02/nomad-bridge-drained-of-nearly-200-million-in-exploit/>.
- Schär, F. (2021). Decentralized finance: On blockchain-and smart contract-based financial markets. *FRB of St. Louis Review*.
- Scharfman, J. (2022). Decentralized finance (DeFi) compliance and operations. In *Cryptocurrency compliance and operations* (pp. 171–186). Springer.
- Shou, C., Tan, S., & Koushik, S. (2023). Ityfuzz: Snapshot-based fuzzer for smart contract. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis ISSTA 2023* (pp. 322–333). New York, NY, USA.
- SlowMist (2023). A brief analysis of the story of the sushi swap attack. Accessed: 08-08-2023 Website. <https://slowmist.medium.com/slowmist-a-brief-analysis-of-the-story-of-the-sushi-swap-attack-c7bc6709adea>.
- So, S., Lee, M., Park, J., Lee, H., & Oh, H. (2020). Verismart: A highly precise safety verifier for Ethereum smart contracts. In *2020 IEEE symposium on security and privacy (SP)* (pp. 1678–1694). IEEE.
- SpartanProtocol (2023). Post mortem analysis. Accessed: 08-08-2023 Website. <https://spartanprotocol.medium.com/today-spartan-protocol-was-subject-to-an-exploit-targeting-the-liquidity-pools-8589b2069cef>.
- Staff, D. H. (2023). yearn.finance reveals details on hack that triggered \$11 million loss. Accessed: 08-08-2023 Website. <https://dailyhodl.com/2021/02/07/yearn-finance-reveals-details-on-hack-that-triggered-11-million-loss/>.
- Su, J., Dai, H.-N., Zhao, L., Zheng, Z., & Luo, X. (2023a). Effectively generating vulnerable transaction sequences in smart contracts with reinforcement learning-guided fuzzing. In *Proceedings of the 37th IEEE/ACM international conference on automated software engineering ASE '22*. New York, NY, USA.
- Su, J., Lin, X., Fang, Z., Zhu, Z., Chen, J., Zheng, Z., Lv, W., & Wang, J. (2023b). Defiwarder: Protecting DeFi apps from token leaking vulnerabilities. In *2023 38th IEEE/ACM International conference on automated software engineering (ASE)* (pp. 1664–1675). IEEE.
- Tann, W. J.-W., Han, X. J., Gupta, S. S., & Ong, Y.-S. (2018). Towards safer smart contracts: A sequence learning approach to detecting security threats. arXiv preprint arXiv:1811.06632.
- Team, p. (2023). Is a new token standard really to blame for the imBTC/uniswap and dforce attacks? Accessed: 08-08-2023 Website. <https://medium.com/pnetwork/is-a-new-token-standard-really-to-blame-for-the-imbtc-uniswap-and-dforce-attacks-31c62e2bc799>.
- THORChain (2023). Certik completes THORChain audit. Accessed: 08-08-2023 Website. <https://medium.com/thorchain/certik-completes-thorchain-audit-c6d88fad3613>.
- Thurman, A. (2023). Cream finance suffers third hack, loses over \$130 million. Accessed: 08-08-2023 Website. <https://decrypt.co/84590/cream-finance-suffers-third-hack-losing-over-130-million>.
- Tian, Z., Luo, C., Qiu, J., Du, X., & Guizani, M. (2020). A distributed deep learning system for web attack detection on edge devices. *IEEE Transactions on Industrial Informatics*, 16(3), 1963–1971. <https://doi.org/10.1109/TII.2019.2938778>
- Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E., & Alexandrov, Y. (2018). Smartcheck: Static analysis of Ethereum smart contracts. In *Proceedings of the 1st international workshop on emerging trends in software engineering for blockchain* (pp. 9–16).
- Tolmach, P., Li, Y., & Lin, S.-W. (2022). Property-based automated repair of DeFi protocols. In *Proceedings of the 37th IEEE/ACM international conference on automated software engineering* (pp. 1–5).
- Tolmach, P., Li, Y., & Lin, S.-W. (2023). Property-based automated repair of DeFi protocols. In *Proceedings of the 37th IEEE/ACM international conference on automated software engineering ASE '22*. New York, NY, USA.
- Tolmach, P., Li, Y., Lin, S.-W., & Liu, Y. (2021). Formal analysis of composable DeFi protocols. In *International conference on financial cryptography and data security* (pp. 149–161). Springer.
- Torres, C. F., Iannillo, A. K., Gervais, A., & State, R. (2021). Confuzzius: A data dependency-aware hybrid fuzzer for smart contracts. In *2021 IEEE european symposium on security and privacy (eurosp)* (pp. 103–119). IEEE.
- Torres, C. F., Schütte, J., & State, R. (2018). Osiris: Hunting for integer bugs in Ethereum smart contracts. In *Proceedings of the 34th annual computer security applications conference* (pp. 664–676).
- Torres, C. F., Steichen, M. et al. (2019). The art of the scam: Demystifying honeypots in Ethereum smart contracts. In *28th USENIX security symposium (USENIX security 19)* (pp. 1591–1607).
- Tsankov, P., Dan, A., Drachler-Cohen, D., Gervais, A., Buenzli, F., & Vechev, M. (2018). Securify: Practical security analysis of smart contracts. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security* (pp. 67–82). ACM.
- Tsankov, P., Dan, A., Drachler-Cohen, D., Gervais, A., Buenzli, F., & Vechev, M. (2023). Securify2. Accessed: 08-08-2023 Website. <https://github.com/eth-sri/securify2>.
- VaultSX (2023). VaultSX hack: Lessons learned and other thoughts. Accessed: 08-08-2023 Website. <https://www.eosgo.io/news/vaultsx-hack-lessons-learned-and-thoughts>.
- Vidal, F. R., Ivaki, N., & Laranjeiro, N. (2023). OpenSCV: An open hierarchical taxonomy for smart contract vulnerabilities. arXiv preprint arXiv:2303.14523.
- Wang, B., Liu, H., Liu, C., Yang, Z., Ren, Q., Zheng, H., & Lei, H. (2021a). Blockeye: Hunting for DeFi attacks on blockchain. In *2021 IEEE/ACM 43rd international conference on software engineering: Companion proceedings (ICSE-companion)* (pp. 17–20). IEEE.
- Wang, B., Yuan, X., Duan, L., Ma, H., Su, C., & Wang, W. (2022a). Defiscanner: Spotting DeFi attacks exploiting logic vulnerabilities on blockchain. *IEEE Transactions on Computational Social Systems*.
- Wang, D., Wu, S., Lin, Z., Wu, L., Yuan, X., Zhou, Y., Wang, H., & Ren, K. (2021b). Towards a first step to understand flash loan and its applications in DeFi ecosystem. In *Proceed-*

- ings of the ninth international workshop on security in blockchain and cloud computing (pp. 23–28).
- Wang, H., Li, Y., Lin, S.-W., Ma, L., & Liu, Y. (2019). Vultron: Catching vulnerable smart contracts once and for all. In *2019 IEEE/ACM 41st international conference on software engineering: New ideas and emerging results (ICSE-NIER)* (pp. 1–4). IEEE.
- Wang, H., Liu, Y., Li, Y., Lin, S.-W., Artho, C., Ma, L., & Liu, Y. (2020a). Oracle-supported dynamic exploit generation for smart contracts. *IEEE Transactions on Dependable and Secure Computing*, 19(3), 1795–1809.
- Wang, S.-H., Wu, C.-C., Liang, Y.-C., Hsieh, L.-H., & Hsiao, H.-C. (2021c). Promutator: Detecting vulnerable price oracles in DeFi by mutated transactions. In *2021 IEEE european symposium on security and privacy workshops (euros&PW)* (pp. 380–385). IEEE.
- Wang, W., Song, J., Xu, G., Li, Y., Wang, H., & Su, C. (2020b). Contractward: Automated vulnerability detection models for Ethereum smart contracts. *IEEE Transactions on Network Science and Engineering*, 8(2), 1133–1144.
- Wang, Y., Lahiri, S. K., Chen, S., Pan, R., Dillig, I., Born, C., & Naseer, I. (2018). Formal specification and verification of smart contracts for azure blockchain. arXiv preprint arXiv:1812.08829.
- Wang, Y., Partick, Z., Yao, Y., Lu, Z., & Wattenhofer, R. (2022b). Impact and user perception of sandwich attacks in the DeFi ecosystem. In *ACM Conference on Human Factors in Computing Systems (CHI)*, New Orleans, LA, USA.
- Wang, Z., Qin, K., Minh, D. V., & Gervais, A. (2022c). Speculative multipliers on DeFi: Quantifying on-chain leverage risks. *Financial Cryptography and Data Security*.
- Wikipedia. (2023). Poly network exploit. Accessed: 08-08-2023 Website. https://en.wikipedia.org/wiki/Poly_Network_exploit.
- Williams, C. (2023a). \$80m lost in attack on rari capital. Accessed: 08-08-2023 Website. <https://cryptobriefing.com/80m-lost-in-attack-on-rari-capital/>.
- Williams, C. (2023b). Alpha finance exploited in \$37.5 million attack. Accessed: 08-08-2023 Website. <https://cryptobriefing.com/alpha-finance-suffers-37-5-million-loss-major-attack/>.
- Williams, I. (2020). Cross-industry use of blockchain technology and opportunities for the future. IGI global.
- Wormhole. (2023). Wormhole incident report. Accessed: 08-08-2023 Website. <https://wormholecrypto.medium.com/wormhole-incident-report-02-02-22-ad9b8f21eec6>.
- Wu, S., Wang, D., He, J., Zhou, Y., Wu, L., Yuan, X., He, Q., & Ren, K. (2021). De-firanger: Detecting price manipulation attacks on DeFi applications. arXiv preprint arXiv:2104.15068.
- Wüstholz, V., & Christakis, M. (2020a). Harvey: A greybox fuzzer for smart contracts. In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering* (pp. 1398–1409).
- Wüstholz, V., & Christakis, M. (2020b). Targeted greybox fuzzing with static lookahead analysis. In *2020 IEEE/ACM 42nd International conference on software engineering (ICSE)* (pp. 789–800). IEEE.
- Xu, W., Moon, H., Kashyap, S., Tseng, P.-N., & Kim, T. (2019). Fuzzing file systems via two-dimensional input space exploration. In *2019 IEEE symposium on security and privacy (SP)* (pp. 818–834). IEEE.
- Xue, Y., Ma, M., Lin, Y., Sui, Y., Ye, J., & Peng, T. (2020). Cross-contract static analysis for detecting practical reentrancy vulnerabilities in smart contracts. In *2020 35th IEEE/ACM International conference on automated software engineering (ASE)* (pp. 1029–1040). IEEE.
- Xue, Y., Ye, J., Zhang, W., Sun, J., Ma, L., Wang, H., & Zhao, J. (2022). xfuzz: Machine learning guided cross-contract fuzzing. *IEEE Transactions on Dependable and Secure Computing*, 21(2), 515–529.
- Ye, J., Ma, M., Lin, Y., Sui, Y., & Xue, Y. (2020). Clairvoyance: Cross-contract static analysis for detecting practical reentrancy vulnerabilities in smart contracts. In *2020 IEEE/ACM 42nd international conference on software engineering: Companion proceedings (ICSE-companion)* (pp. 274–275). IEEE.
- Yu, X., Zhao, H., Hou, B., Ying, Z., & Wu, B. (2021). DeeSCVHunter: A deep learning-based framework for smart contract vulnerability detection. In *2021 international joint conference on neural networks (IJCNN)* (pp. 1–8).
- Yu, X. L., Al-Bataineh, O., Lo, D., & Roychoudhury, A. (2020). Smart contract repair. *ACM Trans. Softw. Eng. Methodol.*, 29(4).
- Zarir, A. A., Oliva, G. A., Jiang, Z. M., & Hassan, A. E. (2021). Developing cost-effective blockchain-powered applications: A case study of the gas usage of smart contract transactions in the Ethereum blockchain platform. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 30(3), 1–38.
- Zhang, Y., Ma, S., Li, J., Li, K., Nepal, S., & Gu, D. (2020). Smartshield: Automatic smart contract protection made easy. In *2020 IEEE 27th international conference on software analysis, evolution and reengineering (SANER)* (pp. 23–34). IEEE.
- Zhou, L., Qin, K., Cully, A., Livshits, B., & Gervais, A. (2021a). On the just-in-time discovery of profit-generating transactions in DeFi protocols. In *2021 IEEE symposium on security and privacy (SP)* (pp. 919–936). IEEE.
- Zhou, L., Qin, K., & Gervais, A. (2021b). A2mm: Mitigating frontrunning, transaction reordering and consensus instability in decentralized exchanges. arXiv preprint arXiv:2106.07371.
- Zhou, L., Qin, K., Torres, C. F., Le, D. V., & Gervais, A. (2021c). High-frequency trading on decentralized on-chain exchanges. In *2021 IEEE symposium on security and privacy (SP)* (pp. 428–445). IEEE.
- Zhou, L., Xiong, X., Ernstberger, J., Chaliasos, S., Wang, Z., Wang, Y., Qin, K., Wattenhofer, R., Song, D., & Gervais, A. (2023). Sok: Decentralized finance (DeFi) attacks. In *2023 IEEE symposium on security and privacy (SP)* (pp. 2444–2461). IEEE.
- Zhuang, Y., Liu, Z., Qian, P., Liu, Q., Wang, X., & He, Q. (2020). Smart contract vulnerability detection using graph neural network. In *Ijcai* (pp. 3283–3290).
- Zou, W., Lo, D., Kochhar, P. S., Le, X.-B. D., Xia, X., Feng, Y., Chen, Z., & Xu, B. (2019). Smart contract development: Challenges and opportunities. *IEEE Transactions on Software Engineering*, 47(10), 2084–2106.